توثيق مشروع SalesOrderManagement

نظام إدارة العملاء وطلبات المبيعات

البند	القيمة
التقنيات	ASP.NET Core MVC و Entity Framework Core و SQL Server
الاسم المستخدم في الكود	`ExamRFID` (اسم Solution و Project و namespace)
اسم التطبيق في الواجهة	Sales Orders
قاعدة البيانات	`SalesOrdersDb` على سيرفر `.\SQLEXPRESS`
إطار العمل	.NET 10 — ASP.NET Core MVC
تاريخ التوثيق	23 أكتوبر 2026

> **ملحوظة عن الاسم:** انت بتسمي المشروع SalesOrderManagement، لكن مفيش أي ملف أو كلاس في الكود بالاسم ده. الاسم الحقيقي في الكود هو `ExamRFID` : ده اسم ملف `ExamRFID.slnx` وملف `ExamRFID.csproj` ، وكل الكلاسات جوه namespace اسمه `ExamRFID` . في التوثيق ده هستخدم الاسمين، بس لما أتكلم عن الكود هكتب الاسم الحقيقي.

المحتويات

1. فكرة المشروع والهدف منه وطريقة عمله
2. هيكل المشروع: كل فولدر وكل ملف مهم
3. شرح كل Model وكل Property
4. العلاقات بين Models وجداول قاعدة البيانات
5. شرح كل Controller وكل Action
6. شرح كل View ومكان ظهورها في المتصفح
7. Routing والتنقل بين الصفحات
8. EF Core و DbContext و Migrations والاتصال بقاعدة البيانات
9. عمليات CRUD: رحلة البيانات من المستخدم لقاعدة البيانات والعكس
10. Validation و Error Handling و Authentication
11. تصميم الواجهة: CSS و JavaScript والمكتبات
12. تتبّع أهم السيناريوهات خطوة بخطوة
13. شرح أهم الأكواد سطر بسطر
14. الأخطاء اللي اتصلحت، والقرارات التقنية، والأجزاء غير المكتملة
15. أسئلة Interview متوقعة مع الإجابات
16. ملحق: قاموس المصطلحات وأوامر مهمة

إزاي تذاكر الملف ده؟ لو وقتك قليل قبل الـ Interview، ركّز على الأقسام 1 و 4 و 8 و 9 و 14 و 15. لو عايز تفهم كل سطر، امشي بالترتيب.

قاعدة مهمة: كل حاجة مكتوبة هنا متأكد منها من الكود الحقيقي للمشروع. ولو فيه حاجة مش ممكن أتأكد منها من الكود (زي البيانات اللي في قاعدة البيانات عندك دلوقتي)، هكتب ده صراحة.

1. فكرة المشروع والهدف منه وطريقة عمله

1.1 الفكرة (Business Scenario)

شركة عندها **عملاء (Customers)**، وكل عميل بيعمل **طلبات مبيعات (Orders)**، وكل طلب فيه **عناصر (Order Items)**، يعني منتجات بكمية وسعر.

الموظفين محتاجين:

- يشوفوا قائمة الطلبات.
- يفتحوا أي طلب ويعرفوا مين العميل اللي عمله، وإمتى اتعمل، وفيه أنهي منتجات.
- يضيفوا ويعدّلوا ويمسحوا العملاء والطلبات والعناصر.
- يلاقوا رسايل واضحة لو البيانات غلط، أو لو السجل اللي بيدوروا عليه مش موجود.

وكمان النظام لازم يفضل **سريع** حتى لو البيانات كترت جداً.

1.2 الهدف من المشروع

المشروع معمول كأساس واقعي لتقييم الحاجات دي:

1. **اختيار طريقة الوصول للبيانات (Data Access):** إزاي نجيب البيانات المرتبطة ببعض (طلب + عميله + عناصره) بأقل عدد استعلامات وأقل بيانات.
2. **الأداء (Performance):** فيه صفحة مخصوصة بتقارن 3 طرق لتحميل البيانات وبتقيس وقت كل طريقة وعدد استعلامات SQL.
3. **Validation:** رفض البيانات الغلط برسايل واضحة.
4. **Error Handling:** التعامل مع الأخطاء من غير ما نعرض تفاصيل تقنية للمستخدم.

1.3 المشروع بيعمل إيه فعلاً؟ (الـ Features الموجودة)

الجزء	اللي موجود فعلاً في الكود
العملاء	قائمة بصفحات (20 عميل في الصفحة)، تفاصيل العميل مع طلباته، إضافة، تعديل، حذف (ممنوع حذف عميل عنده طلبات)
الطلبات	قائمة بصفحات (20 طلب في الصفحة) فيها اسم العميل وعدد العناصر والإجمالي، تفاصيل الطلب مع العميل والعناصر، إضافة، تعديل، حذف (بيمسح العناصر معاه)
عناصر الطلب	إضافة وتعديل وحذف عنصر جوه طلب (مفيش صفحة قائمة منفصلة؛ العناصر بتظهر في صفحة تفاصيل الطلب)
الأداء	مقارنة 3 طرق تحميل، عرض الـ SQL اللي EF Core عمله، حفظ النتايج في ملف CSV، زرار لإنشاء بيانات تجريبية
الأخطاء	صفحة "Not found" لما السجل مش موجود، صفحة Error عامة، رسايل نجاح وخطأ بعد كل عملية

الحاجات اللي مش موجودة (عشان لو اتسألت ما تقولش إنها موجودة):

- مفيش تسجيل دخول (Login) ولا صلاحيات (Authorization).
- مفيش بحث (Search) ولا فلترة ولا ترتيب حسب اختيار المستخدم.
- مفيش API (كل الصفحات HTML بتترسم على السيرفر).
- مفيش Unit Tests.

- مفيش رسومات بيانية (Charts) ولا تقارير.
- مفيش Services ولا Repository Pattern (هشرح السبب في القسم 14).

1.4 المشروع بيشتغل إزاي؟ (الصورة الكبيرة)

المشروع مبني بنمط **MVC**، وده اختصار لـ:

- **Model:** الكلاسات اللي بتمثل البيانات، زي `Customer` و `Order`.
- **View:** صفحات HTML اللي المستخدم بيشوفها (ملفات `.cshtml`).
- **Controller:** الكلاس اللي بيستقبل طلب المتصفح، ويكلم قاعدة البيانات، ويختار أنهي View تتعرض.

الفكرة إن كل جزء له شغلانة واحدة: Controller مش بيرسم HTML، و View مش بتكلم قاعدة البيانات.

رحلة أي طلب (Request) من المتصفح، مثلاً صفحة الطلبات:

```
Browser: GET /Orders
   │
   ▼
Kestrel (the web server built into ASP.NET Core)
   │
   ▼
Middleware pipeline (defined in Program.cs)
   StatusCodePages → RequestLocalization → HttpsRedirection → Routing → Authorization
   │
   ▼
Routing: "/Orders"  →  OrdersController.Index()
   │
   ▼
OrdersController.Index()  ── AppDbContext (EF Core) ──►  SQL Server (SalesOrdersDb)
   │                                                      one SELECT query
   ▼
List<OrderListItem>  →  Views/Orders/Index.cshtml  inside  Views/Shared/_Layout.cshtml
   │
   ▼
HTML goes back to the browser
```

نفس الرحلة بالعربي:

1. المتصفح بيطلب `/Orders`.
2. **Kestrel** (سيرفر الويب اللي جوه ASP.NET Core) بيستقبل الطلب.
3. الطلب بيعدّي على **Middleware Pipeline** بالترتيب المكتوب في `Program.cs`.
4. **Routing** بيعرف إن `/Orders` معناها `OrdersController.Index()`.
5. الـ Controller بيستخدم `AppDbContext` (يعني EF Core) عشان يجيب البيانات من SQL Server في استعلام واحد.
6. الـ Controller بيبعت `List<OrderListItem>` للـ View.
7. الـ View بتترسم جوه الـ Layout، والـ HTML يرجع للمتصفح.

مصطلحات ظهرت هنا:

- **Request:** الطلب اللي المتصفح بيبعته للسيرفر، زي "هات صفحة `/Orders`".

- **Middleware:** خطوة في سلسلة بيعدّي عليها كل Request، وكل خطوة ليها وظيفة (تحويل لـ HTTPS، تحديد الـ Controller المناسب، ...).
- **Pipeline:** السلسلة نفسها، بالترتيب المكتوب في `Program.cs` .
- **EF Core (Entity Framework Core):** مكتبة بتحوّل كود C# لاستعلامات SQL، فمش محتاج تكتب SQL بإيدك.

2. هيكل المشروع: كل فولدر وكل ملف مهم

2.1 شكل الفولدرات

```
ExamRFID/                              <- solution folder
├── ExamRFID.slnx                      <- solution file
└── ExamRFID/                          <- the web project
    ├── Program.cs                     <- start point: services + pipeline
    ├── ExamRFID.csproj                <- project settings + NuGet packages
    ├── ExamRFID.csproj.user           <- personal Visual Studio settings
    ├── appsettings.json               <- connection string + logging
    ├── appsettings.Development.json   <- logging while developing
    ├── Properties/
    │   └── launchSettings.json        <- ports and run profiles
    ├── Controllers/
    │   ├── HomeController.cs
    │   ├── CustomersController.cs
    │   ├── OrdersController.cs
    │   ├── OrderItemsController.cs
    │   └── PerformanceController.cs
    ├── Data/
    │   ├── AppDbContext.cs            <- database connection + Fluent API
    │   └── SeedData.cs                <- creates test data
    ├── Migrations/
    │   ├── 20261020230104_InitialCreate.cs
    │   ├── 20261020230104_InitialCreate.Designer.cs
    │   └── AppDbContextModelSnapshot.cs
    ├── Models/
    │   ├── Customer.cs                <- table
    │   ├── Order.cs                   <- table
    │   ├── OrderItem.cs               <- table
    │   ├── OrderListItem.cs           <- view model: one row of the order list
    │   ├── PerformanceResult.cs       <- view model: result of one strategy
    │   ├── PerformanceViewModel.cs    <- view model: the Performance page
    │   └── ErrorViewModel.cs          <- view model: the error page
    ├── Views/
    │   ├── _ViewImports.cshtml
    │   ├── _ViewStart.cshtml
    │   ├── Shared/      (_Layout, _Pager, _ValidationScriptsPartial, Error)
    │   ├── Home/        (Index, Privacy, HttpError)
    │   ├── Customers/   (Index, Details, Create, Edit, Delete)
    │   ├── Orders/      (Index, Details, Create, Edit, Delete)
    │   ├── OrderItems/  (Create, Edit, Delete)
    │   └── Performance/ (Index)
    ├── wwwroot/
    │   ├── css/site.css               <- all the site design
    │   ├── js/site.js                 <- empty (from the template)
    │   ├── lib/                       <- Bootstrap + jQuery
    │   └── favicon.ico
    ├── perf-results.csv               <- saved performance results
    ├── sales.db                       <- old file, not used (see below)
    ├── Documentation/                 <- these documentation files
    └── bin/ and obj/                  <- build output (created automatically)
```

2.2 الملفات في جذر المشروع

الملف	ليه موجود؟	بيعمل إيه؟	مرتبط بإيه؟
`ExamRFID.slnx`	Visual Studio محتاج ملف Solution يجمّع المشاريع	فيه سطر واحد بيشاور على `ExamRFID/ExamRFID.csproj` . صيغة `.slnx` هي الصيغة الجديدة (XML) بدل `.sln` القديمة	بتفتحه في Visual Studio عـ
`Program.cs`	كل تطبيق ASP.NET Core لازم له نقطة بداية	بيسجّل الخدمات (MVC و DbContext)، بيضيف البيانات التجريبية أول مرة، وبيبني الـ Pipeline	مرتبط بكل حاجة: `Context` `ings.json` و `SeedData`
`ExamRFID.csproj`	ملف المشروع	بيحدد `net10.0` ، ومفعّل `Nullable` و `ImplicitUsings` ، وفيه NuGet 4 packages (تحت)	الـ Build و NuGet
`ExamRFID.csproj.user`	Visual Studio بيعمله تلقائي	بيحفظ إن آخر Profile اخترته هو `https`	Visual Studio بس، ملوش
`appsettings.json`	مكان الإعدادات اللي ممكن تتغير من غير ما نغير الكود	فيه `DefaultConnection` (عنوان قاعدة البيانات) وإعدادات الـ Logging	`troller` و `Program.cs` بيقروا منه الـ `nection String`
`appsettings.Development.json`	إعدادات بتتطبق بس لما البيئة Development	فيه نفس إعدادات الـ Logging	بيتقرا فوق `settings.json`
`Properties/launchSettings.json`	بيحدد إزاي المشروع يشتغل من Visual Studio أو `dotnet run`	Profile اسمه `http` على `http://localhost:5198` ، و Profile اسمه `https` على `https://localhost:7294` . الاتنين بيحطوا `ASPNETCORE_ENVIRONMENT=Development`	البيئة Development هي الـ الأخطاء التفصيلية تظهر
`perf-results.csv`	صفحة الأداء بتحفظ فيه كل نتيجة	كل سطر = طريقة واحدة في اختبار واحد. فيه حالياً اختبارين اتعملوا يوم 21 أكتوبر	`troller.SaveResults()` و `LoadHistory()`
`sales.db`	**ملف قديم مش مستخدم**	ده ملف SQLite من أول نسخة في المشروع قبل ما ننقل لـ SQL Server. الكود دلوقتي مفيهوش أي إشارة له	ممكن تمسحه من غير أي مش

الـ NuGet Packages الموجودة في `ExamRFID.csproj` :

Package	النسخة	وظيفته
`Microsoft.EntityFrameworkCore.SqlServer`	10.0.13	الـ Provider اللي بيخلي EF Core يكلم SQL Server (فيه `UseSqlServer`)
`Microsoft.EntityFrameworkCore.Tools`	10.0.13	بيضيف أوامر `Add-Migration` و `Update-Database` في Package Manager Console

Package	النسخة	وظيفته
`Microsoft.EntityFrameworkCore.Design`	10.0.13	محتاجينه وقت التصميم عشان أدوات الـ Migrations تقدر تبني الـ DbContext
`System.Security.Cryptography.Pkcs`	10.0.13	مش بنستخدمه مباشرة. ضفناه عشان نسخة 9.0.11 اللي كانت جاية مع SQL Server driver فيها ثغرات أمنية معروفة (شرح في القسم 14)

كلمة NuGet: هو مخزن المكتبات الجاهزة في .NET.، زي npm في JavaScript.

2.3 فولدر Controllers

كل ملف هنا كلاس بيورث من `Controller` ، وكل method عامة (public) فيه اسمها **Action**، والـ Action بترد على رابط معين.

الملف	المسؤول عن
`HomeController.cs`	الصفحة الرئيسية (Dashboard)، صفحة Privacy، صفحة الخطأ 404، صفحة الخطأ العامة
`CustomersController.cs`	كل صفحات العملاء + فحص إن الإيميل مش متكرر
`OrdersController.cs`	كل صفحات الطلبات + `CheckOrderAsync` (فحوصات زيادة) و `FillCustomerListAsync`
`OrderItemsController.cs`	إضافة وتعديل وحذف عناصر الطلب
`PerformanceController.cs`	صفحة مقارنة الأداء، وإنشاء البيانات التجريبية، وحفظ النتايج في CSV

2.4 فولدر Data

الملف	ليه موجود؟
`AppDbContext.cs`	ده "الجسر" بين الكود وقاعدة البيانات. فيه الـ 3 جداول (`DbSet`) وكل قواعد الجداول بالـ Fluent API
`SeedData.cs`	فيه method واحدة `AddTestData` بتعمل عملاء وطلبات وعناصر وهمية. بتتنده من `Program.cs` أول مرة (100 عميل)، ومن زرار Generate في صفحة الأداء

2.5 فولدر Migrations

الفولدر ده اتعمل **تلقائي** بأمر `Add-Migration InitialCreate` (أو `dotnet ef migrations add InitialCreate`). ما حدش كتبه بإيده.

الملف	بيعمل إيه؟
`20261020230104_InitialCreate.cs`	فيه method اسمها `Up` بتعمل الـ 3 جداول والـ Indexes، و method اسمها `Down` بتمسحهم (عشان لو حبيت ترجع في الـ Migration)
`20261020230104_InitialCreate.Designer.cs`	صورة من الـ Model وقت عمل الـ Migration دي (EF بيستخدمها داخلياً)
`AppDbContextModelSnapshot.cs`	صورة من الـ Model **الحالي**. لما تعمل Migration جديدة، EF بيقارن الكود بالـ Snapshot ده عشان يعرف إيه اللي اتغير

الرقم `20261020230104` هو تاريخ ووقت عمل الـ Migration (UTC): 2026-10-20 الساعة 23:01:04.

2.6 فولدر Models

فيه نوعين من الكلاسات:

1. **Entities (كيانات):** `Customer` و `Order` و `OrderItem`. كل واحد منهم بيمثل **جدول** في قاعدة البيانات.
2. **View Models:** `OrderListItem` و `PerformanceResult` و `PerformanceViewModel` و `ErrorViewModel`. دول **مش جداول**؛ دول كلاسات بتشيل البيانات اللي صفحة معينة محتاجاها بس.

2.7 فولدر Views

كل فولدر جوه `Views` اسمه نفس اسم الـ Controller من غير كلمة "Controller". ASP.NET Core بيدور على الـ View في المكانين دول بالترتيب:

1. `Views/{اسم الـ Controller}/{اسم الـ Action}.cshtml`
2. `Views/Shared/{اسم الـ Action}.cshtml`

ده بالظبط سبب الـ Error اللي ظهر قبل كده لما فولدر `Orders` اتنقل بالغلط جوه `OrderItems` (شرح في القسم 14).

الملف	وظيفته
`_ViewStart.cshtml`	بيتنفذ قبل كل View، وبيحدد إن الـ Layout هو `_Layout`
`_ViewImports.cshtml`	بيضيف `@using ExamRFID` و `@using ExamRFID.Models` و Tag Helpers لكل الـ Views، عشان ما نكررهمش في كل ملف
`Shared/_Layout.cshtml`	الإطار الثابت لكل الصفحات: الـ Sidebar، رسايل النجاح والخطأ، وملفات CSS و JS
`Shared/_Pager.cshtml`	جزء صغير (Partial View) فيه زرار Previous و Next، بيتعاد استخدامه في قائمة العملاء والطلبات
`Shared/_ValidationScriptsPartial.cshtml`	بيضيف سكريبتات jQuery Validation اللي بتتحقق من الفورم في المتصفح. مستخدم في كل الفورمز وفي صفحة الأداء
`Shared/Error.cshtml`	صفحة الخطأ العامة (Production بس)

2.8 فولدر wwwroot

ده الفولدر **الوحيد** اللي ملفاته بتتبعت للمتصفح زي ما هي (Static Files).

المسار	المحتوى
`css/site.css`	كل تصميم الموقع الغامق (587 سطر، وفوق كل جزء تعليق)
`js/site.js`	فاضي، فيه تعليقات القالب بس. مفيش JavaScript خاص بالمشروع
`lib/bootstrap`	Bootstrap 5.3.3 (تصميم وأدوات CSS جاهزة)
`lib/jquery`	jQuery 3.7.1
`lib/jquery-validation` و `lib/jquery-validation-unobtrusive`	jQuery Validation 1.21.0 و Unobtrusive 4.0.0: مكتبات التحقق في المتصفح (مستخدمة في كل الفورمز)
`favicon.ico`	أيقونة التاب في المتصفح (من القالب)

2.9 فولدرات `bin` و `obj`

دول بيتعملوا تلقائي وقت الـ Build. فيهم ملفات الـ DLL والـ EXE والملفات المؤقتة. **ما تعدّلش فيهم**، ومش محتاج ترفعهم على Git.

2.10 التعليقات في الكود

فوق كل Class وكل Method تعليق إنجليزي قصير، غالباً سطرين:

- **السطر الأول:** الـ Method بتعمل إيه، ومعاه الرابط لو هي Action.
- **السطر التاني (`Used by:`):** بتتنده منين، يعني أنهي زرار أو لينك أو Method تانية بتستخدمها.

مفيش تعليقات جوه الـ Methods نفسها. مثال حقيقي من `CustomersController` :

```
// GET /Customers/Details/5 - shows one customer with their orders (newest first).
// Used by: the eye icon in the customer list, and after create/edit.
public async Task<IActionResult> Details(int? id)
```

وكل View أول سطر فيها تعليق Razor بيقول الصفحة دي إيه وأنهي Action بتعرضها، مثلاً:

```
@* The customer list (/Customers). Shown by CustomersController.Index. *@
```

الفولدر `Migrations` ما اتلمسش (EF بيعمله لوحده)، وملف `site.css` فيه تعليق فوق كل جزء.

3. شرح كل Model وكل Property

يعني إيه Model؟ كلاس #C عادي (اسمه كمان **POCO**: Plain Old CLR Object)، فيه Properties بس. EF Core بيحوّل كل Entity منهم لجدول، وكل Property لعمود.

يعني إيه Property؟ متغير جوه الكلاس ليه `get` و `set`، زي `public string Name { get; set; }`.

> **ملحوظة مهمة:** كل Model فيه حاجتين في مكانين مختلفين:
>
> - **Attributes** فوق الـ Properties (زي `[Required]` و `[Range]`): وظيفتها **التحقق من الفورم** وكتابة رسايل الخطأ، في المتصفح وعلى السيرفر.
> - **قواعد قاعدة البيانات** (الجداول، والـ Keys، والعلاقات، وأطوال الأعمدة): متكتبة بالـ **Fluent API** في `AppDbContext`.

الـ Attributes المستخدمة (من namespace اسمه `System.ComponentModel.DataAnnotations`):

Attribute	معناه	مثال من المشروع
`[Required(ErrorMessage = "...")]`	الحقل إجباري، ولو فاضي تظهر الرسالة دي	`Name` و `Email` و `ProductName`
`[StringLength(100)]`	أقصى عدد حروف	100 لـ `Name`، 200 لـ `Email`، 30 لـ `Phone`
`[EmailAddress]`	لازم يكون شكله إيميل	`Email`
`[Phone]`	لازم يكون شكله رقم تليفون	`Phone`
`[Range(1, 1000)]`	الرقم لازم يكون بين رقمين	`Quantity` و `UnitPrice` و `CustomerId`
`[Display(Name = "...")]`	النص اللي بيظهر في الـ Label	"Phone number" و "Unit price"
`[DataType(DataType.Date)]`	خانة تاريخ (Date Picker من غير وقت)	`OrderDate`

اسمها **Data Annotations** (يعني "ملاحظات على البيانات"). ميزتها إن نفس الـ Attribute بيتحقق في **المتصفح** قبل ما الفورم يتبعت، وعلى **السيرفر** كمان.

3.1 الـ Entity: `Customer` (ملف `Models/Customer.cs`)

```
public class Customer
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Please enter the customer's name.")]
    [StringLength(100, ErrorMessage = "The name can be at most 100 characters.")]
    public string Name { get; set; } = "";

    [Required(ErrorMessage = "Please enter an email address.")]
    [EmailAddress(ErrorMessage = "Please enter a valid email address.")]
    [StringLength(200, ErrorMessage = "The email can be at most 200 characters.")]
    public string Email { get; set; } = "";

    [Phone(ErrorMessage = "Please enter a valid phone number.")]
    [StringLength(30, ErrorMessage = "The phone number can be at most 30 characters.")]
    [Display(Name = "Phone number")]
    public string? Phone { get; set; }

    public List<Order> Orders { get; set; } = new List<Order>();
}
```

(الكود ده من غير التعليقات اللي في الملف الأصلي.)

Property	النوع	في قاعدة البيانات	ليه موجودة؟
`Id`	`int`	`int`، Primary Key، Identity	رقم مميز لكل عميل. EF بيعرف إنه الـ Primary Key من الاسم `Id`، وكمان متحدد صراحة بـ `HasKey`. Identity يعني SQL Server هو اللي بيدّي الرقم تلقائي (1، 2، 3...)
`Name`	`string`	`nvarchar(100) NOT NULL`	اسم العميل، بيظهر في كل القوائم. إجباري، وأقصى طول 100 حرف
`Email`	`string`	`nvarchar(200) NOT NULL` + Unique Index	للتواصل، وكمان عشان نميّز العملاء. ممنوع عميلين يبقى ليهم نفس الإيميل
`Phone`	`string?`	`nvarchar(30) NULL`	اختياري. علامة `?` معناها إن القيمة مسموح تكون `null` (فاضية)
`Orders`	`List<Order>`	**مش عمود**	**Navigation Property**: قائمة طلبات العميل. EF بيملاها لما نطلب ده بـ `Include`

ليه `= ""` بعد `Name` و `Email`؟ لأن المشروع مفعّل **Nullable Reference Types** (`<Nullable>enable</Nullable>` في `.csproj`). يعني الكومبايلر بيحذّرك لو فيه `string` ممكن يبقى `null` من غير ما تقول. `= ""` بتدّيها قيمة بداية فاضية، فالتحذير بيختفي.

ليه `= new List<Order>()`؟ عشان القائمة ما تبقاش `null` أبداً. لو عملت `customer.Orders.Count` على عميل جديد، هترجع 0 بدل ما يحصل `NullReferenceException`.

يعني إيه Navigation Property؟ Property بتشاور على Entity تانية مرتبطة. مش بتتخزن كعمود؛ EF بيستخدمها عشان يربط الجداول ببعض. مثال: `customer.Orders` بتجيب طلبات العميل، و `order.Customer` بتجيب العميل صاحب الطلب.

3.2 الـ Entity: `Order` (ملف `Models/Order.cs`)

```
public class Order
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Please enter the order date.")]
    [DataType(DataType.Date)]
    [Display(Name = "Order date")]
    public DateTime OrderDate { get; set; } = DateTime.Today;

    [Range(1, int.MaxValue, ErrorMessage = "Please choose a customer.")]
    [Display(Name = "Customer")]
    public int CustomerId { get; set; }

    public Customer? Customer { get; set; }
    public List<OrderItem> Items { get; set; } = new List<OrderItem>();
}
```

(الكود من غير التعليقات اللي في الملف الأصلي.)

ليه `[Range(1, int.MaxValue)]` على `CustomerId`؟ أول اختيار في الـ Dropdown هو "-- Choose a customer --" وقيمته 0. الـ Range بيقول: لازم القيمة تكون 1 أو أكتر، يعني لازم تختار عميل حقيقي.

Property	النوع	في قاعدة البيانات	ليه موجودة؟
`Id`	`int`	Primary Key، Identity	رقم الطلب، بيظهر في الواجهة كـ `#5856` مثلاً
`OrderDate`	`DateTime`	`datetime2 NOT NULL` + Index	تاريخ الطلب. القيمة الافتراضية النهارده (`DateTime.Today`)، عشان فورم الإضافة يبدأ بتاريخ اليوم. عليه Index لأن القائمة بتترتب بيه
`CustomerId`	`int`	`int NOT NULL`، **Foreign Key** لجدول `Customers`	ده العمود اللي بيربط الطلب بالعميل. الـ Dropdown في الفورم بيبعت قيمته
`Customer`	`Customer?`	**مش عمود**	Navigation Property للعميل. عليها `?` لأنها بتبقى `null` لحد ما نعمل `Include(o => o.Customer)`
`Items`	`List<OrderItem>`	**مش عمود**	Navigation Property لعناصر الطلب

يعني إيه Foreign Key؟ عمود في جدول بيشاور على الـ Primary Key في جدول تاني. هنا `Orders.CustomerId` بيشاور على `Customers.Id`. وقاعدة البيانات نفسها بتمنع إنك تحط `CustomerId` لعميل مش موجود.

ليه `Customer?` عليها `?` بينما `CustomerId` لأ؟ لأن الطلب لازم يبقى له عميل في قاعدة البيانات (`CustomerId` إجباري)، لكن **الكائن** `Customer` جوه الكود مش بيتحمّل تلقائي. لو ما عملتش `Include`، هتلاقيه `null`. ولو ما كانش عليه `?`، كان MVC هيعتبره حقل إجباري في الفورم.

ملحوظة: مفيش عمود للإجمالي (Total) في جدول `Orders`. الإجمالي بيتحسب كل مرة من العناصر (`Quantity × UnitPrice`). كده مفيش خطر إن الإجمالي المتخزن يختلف عن العناصر الحقيقية.

3.3 الـ Entity: `OrderItem` (ملف `Models/OrderItem.cs`)

```
public class OrderItem
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Please enter the product name.")]
    [StringLength(100, ErrorMessage = "The product name can be at most 100 characters.")]
    [Display(Name = "Product")]
    public string ProductName { get; set; } = "";

    [Required(ErrorMessage = "Please enter the quantity.")]
    [Range(1, 1000, ErrorMessage = "Quantity must be between 1 and 1000.")]
    public int Quantity { get; set; } = 1;

    [Required(ErrorMessage = "Please enter the unit price.")]
    [Range(0.01, 100000, ErrorMessage = "Unit price must be between 0.01 and 100,000.")]
    [Display(Name = "Unit price")]
    public decimal UnitPrice { get; set; }

    public int OrderId { get; set; }
    public Order? Order { get; set; }
}
```

Property	النوع	في قاعدة البيانات	ليه موجودة؟
`Id`	`int`	Primary Key، Identity	رقم العنصر، بيستخدم في روابط Edit و Delete
`ProductName`	`string`	`nvarchar(100) NOT NULL`	اسم المنتج. مفيش جدول منتجات منفصل؛ الاسم بيتكتب نص (قرار للتبسيط)
`Quantity`	`int`	`int NOT NULL`	الكمية. القيمة الافتراضية 1، والتحقق بيسمح من 1 لـ 1000
`UnitPrice`	`decimal`	`decimal(18,2) NOT NULL`	سعر القطعة الواحدة. التحقق بيسمح من 0.01 لـ 100,000
`OrderId`	`int`	Foreign Key لجدول `Orders`	بيربط العنصر بالطلب
`Order`	`Order?`	**مش عمود**	Navigation Property للطلب

ليه `decimal` مش `double` للسعر؟ `double` بيخزن الأرقام بطريقة تقريبية (Binary)، فممكن `0.1 + 0.2` تطلع `0.30000000000000004`. `decimal` دقيق في الأرقام العشرية، وده المطلوب للفلوس.

يعني إيه `decimal(18,2)`؟ أقصى عدد أرقام 18، منهم 2 بعد العلامة العشرية. يعني أكبر رقم ممكن 9999999999999999.99.

3.4 الـ View Model: `OrderListItem` (ملف `Models/OrderListItem.cs`)

```
public class OrderListItem
{
    public int Id { get; set; }
    public DateTime OrderDate { get; set; }
    public string CustomerName { get; set; } = "";
    public int ItemCount { get; set; }
    public decimal Total { get; set; }
}
```

يعني إيه View Model؟ كلاس مصمم للصفحة مش لقاعدة البيانات. فيه بالظبط اللي الصفحة هتعرضه، ومش ليه جدول ولا `DbSet`.

ليه موجود؟ قائمة الطلبات محتاجة: رقم الطلب، التاريخ، **اسم** العميل، **عدد** العناصر، و**الإجمالي**. لو حمّلنا `Order` كامل مع العميل وكل العناصر، هنجيب بيانات كتير مش محتاجينها. بدل كده بنعمل **Projection** (بالـ `Select`)، فقاعدة البيانات بترجع صف صغير واحد لكل طلب، وهي اللي بتعدّ العناصر وتجمع الإجمالي.

مستخدم فين؟ في `OrdersController.Index` (القائمة)، وفي `PerformanceController` (الطرق التلاتة كلها بترجع `List<OrderListItem>` عشان نقارنهم ببعض).

3.5 الـ View Model: `PerformanceResult` (ملف `Models/PerformanceResult.cs`)

بيشيل نتيجة طريقة واحدة من الطرق التلاتة في صفحة الأداء.

Property	النوع	معناها
`Strategy`	`string`	اسم الطريقة، زي `"Include (eager loading)"`
`AverageMs`	`double`	متوسط الوقت بالملّي ثانية لكل التشغيلات المقيسة
`FastestMs`	`double`	أسرع تشغيلة
`SlowestMs`	`double`	أبطأ تشغيلة
`QueryCount`	`int`	عدد استعلامات SQL في التشغيلة الواحدة (بنعدّه بإيدينا في الكود)
`OrdersLoaded`	`int`	عدد الطلبات اللي اتحمّلت
`GrandTotal`	`decimal`	مجموع إجماليات كل الطلبات
`SameDataAsFirst`	`bool`	`true` لو الطريقة دي رجّعت نفس بيانات أول طريقة (للتأكد إن كلهم صح)
`Sql`	`string`	نص الـ SQL اللي EF Core عمله للاستعلام الأساسي (من `ToQueryString()`)

3.6 الـ View Model: `PerformanceViewModel` (ملف `Models/PerformanceViewModel.cs`)

كل اللي صفحة الأداء محتاجاه في كلاس واحد:

```
[Range(1, 5000, ErrorMessage = "Orders to load must be between 1 and 5000.")]
[Display(Name = "Orders to load")]
public int OrdersToLoad { get; set; } = 200;

[Range(1, 50, ErrorMessage = "Runs must be between 1 and 50.")]
[Display(Name = "Measured runs per strategy")]
public int Runs { get; set; } = 5;

[Range(10, 5000, ErrorMessage = "Customers must be between 10 and 5000.")]
[Display(Name = "Customers to generate")]
public int CustomersToGenerate { get; set; } = 1000;
```

وفيه كمان: `CustomerCount` و `OrderCount` و `ItemCount` (عدد الصفوف في الجداول)، و `Results` (نتايج الاختبار الحالي)، و `History` (النتايج القديمة من ملف CSV، كل سطر `string[]`)، و `Message` (رسالة تحذير لو ملف CSV ما اتحفظش).

زي باقي الـ Models، الـ Attributes هنا وظيفتها التحقق من الفورم وكتابة اسم الـ Label. والكلاس ده **مش جدول** في قاعدة البيانات، فملوش أي قواعد في الـ Fluent API:

- `[Range]` : القيمة لازم تكون بين رقمين.
- `[Display(Name = ...)]` : النص اللي بيظهر في الـ Label.

3.7 الـ View Model: `ErrorViewModel` (ملف `Models/ErrorViewModel.cs`)

جاي من قالب المشروع:

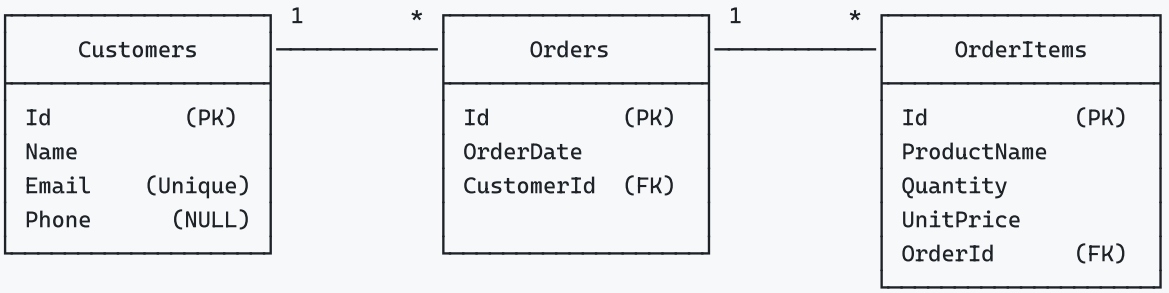
```
public class ErrorViewModel
{
    public string? RequestId { get; set; }
    public bool ShowRequestId => !string.IsNullOrEmpty(RequestId);
}
```

- `RequestId` : رقم مميز للطلب اللي حصل فيه الخطأ. بيظهر للمستخدم عشان لو كلّم الدعم الفني، المطوّر يقدر يلاقي الخطأ في الـ Log.
- `ShowRequestId` : Property محسوبة (مفيهاش `set`)، بترجع `true` لو فيه `RequestId`. علامة `=>` اسمها **Expression-bodied member**، يعني الـ Property دي قيمتها هي نتيجة التعبير ده.

4. العلاقات بين Models وجداول قاعدة البيانات

4.1 شكل العلاقات (ERD)

ERD (Entity Relationship Diagram): رسمة بتوضح الجداول والعلاقات بينها.



حذف العميل ممنوع لو عنده طلبات (RESTRICT)، وحذف الطلب بيمسح عناصره معاه (CASCADE).

- **عميل واحد ← طلبات كتير (One-to-Many):** كل طلب له عميل واحد بس، والعميل ممكن يبقى له صفر أو أكتر من الطلبات.
- **طلب واحد ← عناصر كتير (One-to-Many):** كل عنصر تابع لطلب واحد، والطلب ممكن يبقى فيه صفر أو أكتر من العناصر.

4.2 الجداول بالتفصيل (من ملف الـ Migration)

جدول `Customers` :

العمود	النوع في SQL Server	Null؟	ملاحظات
`Id`	`int`	لا	Primary Key، `IDENTITY(1,1)`
`Name`	`nvarchar(100)`	لا	عليه Index اسمه `IX_Customers_Name`
`Email`	`nvarchar(200)`	لا	عليه Index **Unique** اسمه `IX_Customers_Email`
`Phone`	`nvarchar(30)`	نعم	

جدول `Orders` :

العمود	النوع في SQL Server	Null؟	ملاحظات
`Id`	`int`	لا	Primary Key، `IDENTITY(1,1)`
`OrderDate`	`datetime2`	لا	عليه Index اسمه `IX_Orders_OrderDate`
`CustomerId`	`int`	لا	Foreign Key اسمه `FK_Orders_Customers_CustomerId` (RESTRICT)، وعليه Index اسمه `IX_Orders_CustomerId`

جدول `OrderItems` :

العمود	النوع في SQL Server	Null؟	ملاحظات
`Id`	`int`	لا	Primary Key، `IDENTITY(1,1)`
`ProductName`	`nvarchar(100)`	لا	
`Quantity`	`int`	لا	
`UnitPrice`	`decimal(18,2)`	لا	
`OrderId`	`int`	لا	Foreign Key اسمه `FK_OrderItems_Orders_OrderId` (CASCADE)، وعليه Index اسمه `IX_OrderItems_OrderId`

وفيه جدول رابع بيعمله EF Core لوحده: `__EFMigrationsHistory` ، وفيه أسماء الـ Migrations اللي اتطبقت على قاعدة البيانات (حالياً سطر واحد: `20261020230104_InitialCreate`).

مصطلحات:

- `nvarchar(n)` **:** نص يدعم كل اللغات (Unicode)، وأقصى طوله n حرف.
- SQL Server **Identity:** بيولّد الرقم تلقائي مع كل صف جديد. `(1,1)` يعني يبدأ من 1 ويزيد 1.
- **Index:** زي فهرس الكتاب. بيخلي البحث والترتيب بعمود معين أسرع بكتير، خصوصاً لما البيانات تكتر.
- Index **Unique Index:** كمان بيمنع تكرار القيمة.

4.3 العلاقات متعرّفة فين وإزاي؟

في `Data/AppDbContext.cs` ، جوه method اسمها `OnModelCreating` . ده الكود الحقيقي لعلاقة العميل بالطلبات (سطور 60 لـ 63):

```
entity.HasOne(o => o.Customer)          // كل طلب له عميل واحد
    .WithMany(c => c.Orders)            // والعميل له طلبات كتير
    .HasForeignKey(o => o.CustomerId)   // وده عمود الربط بينهم
    .OnDelete(DeleteBehavior.Restrict); // وممنوع تمسح عميل عنده طلبات
```

وعلاقة الطلب بالعناصر (سطور 83 لـ 86):

```
entity.HasOne(i => i.Order)             // كل عنصر تابع لطلب واحد
    .WithMany(o => o.Items)             // والطلب فيه عناصر كتير
    .HasForeignKey(i => i.OrderId)      // وده عمود الربط بينهم
    .OnDelete(DeleteBehavior.Cascade);  // ولو الطلب اتمسح، العناصر تتمسح معاه
```

(التعليقات العربي متضافة هنا للشرح بس. في الملف الأصلي مفيش تعليقات جوه `OnModelCreating` ، فيه تعليق إنجليزي واحد فوقها.)

4.4 ليه Restrict هنا و Cascade هناك؟

العلاقة	الاختيار	السبب
Customer ← Orders	`Restrict`	الطلبات بيانات مهمة (سجل مبيعات). لو حد مسح عميل بالغلط، مش عايزين كل طلباته تضيع. فقاعدة البيانات **بترفض** الحذف لو العميل عنده طلبات

العلاقة	الاختيار	السبب
Order ← OrderItems	`Cascade`	العنصر ملوش أي معنى من غير الطلب بتاعه. فلما الطلب يتمسح، عناصره تتمسح تلقائي

وزيادة على قاعدة البيانات، الكود نفسه بيتأكد قبل الحذف. `CustomersController.DeleteConfirmed` بيعمل `AnyAsync(o => o.CustomerId == id)` ، ولو لقى طلبات، بيعرض رسالة واضحة بدل ما يسيب قاعدة البيانات ترمي Exception.

4.5 Indexes ليه موجودة؟

Index	السبب
`IX_Customers_Email` (Unique)	يمنع تكرار الإيميل حتى لو طلبين وصلوا في نفس اللحظة وعدّوا من تحقق الكود
`IX_Customers_Name`	قائمة العملاء بتترتب بالاسم (`OrderBy(c => c.Name)`)
`IX_Orders_OrderDate`	قائمة الطلبات بتترتب بالأحدث (`OrderByDescending(o => o.OrderDate)`)، ومعاها Paging. من غير Index، SQL Server هيرتب كل الطلبات في كل مرة
`IX_Orders_CustomerId` و `IX_OrderItems_OrderId`	EF Core بيعمل Index تلقائي على كل Foreign Key، عشان الـ JOIN وجلب طلبات عميل أو عناصر طلب يبقوا سريعين

5. شرح كل Controller وكل Action

5.1 مفاهيم لازم تعرفها الأول

Controller: كلاس اسمه بيخلص بكلمة `Controller` وبيورث من `Controller` . ASP.NET Core بيلاقيه لوحده.

Action: أي method عامة (public) جوه Controller. مثلاً `OrdersController.Index` بترد على `/Orders` .

`IActionResult` : نوع الحاجة اللي الـ Action بترجعها، وبيقول للـ Framework يعمل إيه:

اللي بترجعه	معناه
`View(model)`	اعرض الـ View اللي بنفس اسم الـ Action وابعتلها الـ model
`View("Index", model)`	اعرض View اسمها `Index` (حتى لو اسم الـ Action مختلف)
`NotFound()`	رجّع كود 404 (مش موجود)
`RedirectToAction(...)`	قول للمتصفح يروح على رابط تاني (كود 302)

Dependency Injection (DI): بدل ما الـ Controller يعمل `new AppDbContext()` بنفسه، بيطلبه في الـ Constructor، و ASP.NET Core هو اللي بيدّيهوله:

```
public CustomersController(AppDbContext db, ILogger<CustomersController> logger)
{
    _db = db;
    _logger = logger;
}
```

الفايدة: الـ Controller مش مسؤول عن إنشاء الاتصال ولا قفله. و ASP.NET Core بيعمل `AppDbContext` جديد لكل Request وبيقفله في الآخر (ده اسمه **Scoped lifetime**، وهو الافتراضي لـ `AddDbContext`).

`async` و `await` و `Task` : لما الكود يستنى قاعدة البيانات، `await` بتسيب الـ Thread يخدم Requests تانية لحد ما الرد يوصل. ده بيخلي السيرفر يستحمل مستخدمين أكتر. عشان كده كل methods قاعدة البيانات اسمها بيخلص بـ `Async` (زي `ToListAsync` و `SaveChangesAsync`).

الـ Attributes اللي فوق الـ Actions:

Attribute	معناه
`[HttpPost]`	الـ Action دي بترد على POST بس (لما الفورم يتبعت). من غيرها بترد على GET
`[ValidateAntiForgeryToken]`	لازم الفورم يكون فيه Token سري جاي من موقعنا. ده بيحمي من هجوم اسمه **CSRF** (موقع تاني يبعت فورم باسمك)
`[Bind("Name,Email,Phone")]`	خد الحقول دي بس من الفورم واتجاهل أي حاجة تانية. بيحمي من **Overposting** (حد يبعت حقل زي `Id` من نفسه)
`[ActionName("Delete")]`	خلي الـ method دي ترد على اسم `Delete` حتى لو اسمها `DeleteConfirmed`
`[ResponseCache(...NoStore = true)]`	ما تخليش المتصفح يحتفظ بنسخة من الصفحة (على صفحة Error بس)

`ModelState` **:** قاموس فيه نتيجة التحقق من بيانات الفورم. `ModelState.IsValid` بترجع `false` لو فيه أي خطأ. و `ModelState.AddModelError("Name", "...")` بتضيف رسالة خطأ تحت حقل `Name` .

`TempData` **:** مكان بتحط فيه رسالة تعيش **لطلب واحد بعد الـ Redirect** (متخزنة في Cookie). بنستخدمه لرسايل "The customer was created.".

`ViewBag` **:** طريقة سريعة تبعت بيانات زيادة للـ View من غير ما تحطها في الـ Model، زي رقم الصفحة أو قائمة العملاء للـ Dropdown.

5.2 `HomeController`

Action	Method	الرابط	امتى بيتنده؟	بيعمل إيه؟
`Index()`	GET	`/` أو `/Home` أو `/Home/Index`	لما تفتح الموقع، أو تدوس Dashboard أو اسم الموقع في الـ Sidebar	بيعرض `Views/Home/Index.cshtml` (3 مربعات بروابط)
`Privacy()`	GET	`/Home/Privacy`	لينك Privacy تحت في الـ Sidebar	بيعرض صفحة فيها نص مبدئي من القالب
`HttpError(int code)`	أي Method	`/Home/HttpError?code=404`	**مش المستخدم اللي بيندهه.** Middleware اسمه `UseStatusCodePagesWithReExecute` بيندهه لما أي Action ترجع 404 مثلاً	بيحط `code` في `ViewBag.Code` ويعرض صفحة "Not found"
`Error()`	أي Method	`/Home/Error`	بيتنده تلقائي لما يحصل Exception مش متوقع، **في Production بس** (`UseExceptionHandler`)	بيعرض صفحة خطأ عامة فيها `RequestId` بس، من غير أي تفاصيل تقنية

5.3 `CustomersController`

Action	Method	الرابط	امتى بيتنده؟	بيعمل إيه؟	النتيجة
`Index(int page = 1)`	GET	`/Customers?page=2`	Customers في الـ Sidebar، أو Previous/Next	بيعد العملاء، بيحسب عدد الصفحات، بيجيب 20 عميل بس مترتبين بالاسم	`View(customers)`
`Details(int? id)`	GET	`/Customers/Details/5`	أيقونة العين في القائمة	بيجيب العميل **وطلباته** (الأحدث الأول) في استعلام واحد بـ `Include`	`View(customer)` أو 404
`Create()`	GET	`/Customers/Create`	زرار "New customer"	بيعرض فورم فاضي	`View()`
`Create(Customer)`	POST	`/Customers/Create`	لما تدوس Save في فورم الإضافة	الـ Attributes بتتحقق من الحقول، وبعدين بيتأكد إن الإيميل مش متكرر، يضيف، يحفظ، ويحوّلك لصفحة التفاصيل	Redirect، أو نفس الفورم بالأخطاء

Action	Method	الرابط	امتى بيتنده؟	بيعمل إيه؟	النتيجة
`Edit(int? id)`	GET	`/Customers/Edit/5`	أيقونة القلم أو زرار Edit	بيجيب العميل بـ `FindAsync` ويعرضه في الفورم	`ew(customer)` أو 404
`Edit(int id, Customer)`	POST	`/Customers/Edit/5`	Save في فورم التعديل	بيجيب العميل المحفوظ، يتأكد إن الإيميل مش مستخدم عند عميل **تاني**، ينسخ القيم الجديدة عليه، ويحفظ	Redirect للتفاص أو الفورم بالأخطاء
`Delete(int? id)`	GET	`/Customers/Delete/5`	أيقونة الزبالة أو زرار Delete	بيعرض صفحة "متأكد؟" وبيعد طلبات العميل (`ViewBag.OrderCount`)	`ew(customer)` أو 404
`DeleteConfirmed(int id)`	POST	`/Customers/Delete/5`	"Yes, delete"	لو العميل عنده طلبات: رسالة خطأ ورجوع. غير كده: يمسح ويرجّعك للقائمة	Redirect

5.4 `OrdersController`

Action	Method	الرابط	امتى بيتنده؟	بيعمل إيه؟	النتيجة
`Index(int page = 1)`	GET	`/Orders?page=2`	Orders في الـ Sidebar	بيجيب 20 طلب (الأحدث الأول) بـ **Projection** لـ `OrderListItem` في استعلام واحد	`View(orders)`
`Details(int? id)`	GET	`/Orders/Details/5`	أيقونة العين، أو بعد أي إضافة/تعديل	بيجيب الطلب **+ العميل + العناصر** بـ `Include` و `AsNoTracking`	`View(order)` أو 404
`Create(int? customerId)`	GET	`/Orders/Create` أو `/Orders/Create?customerId=5`	"New order"، أو "New order for this customer" من صفحة العميل	بيعمل طلب جديد بتاريخ النهارده، ولو فيه `customerId` بيختار العميل في الـ Dropdown	`View(order)`
`Create(Order)`	POST	`/Orders/Create`	Save	بيتحقق (`CheckOrderAsync`)، يحفظ، ويحوّلك للتفاصيل عشان تضيف عناصر	Redirect، أو الفورم بالأخطاء
`Edit(int? id)`	GET	`/Orders/Edit/5`	Edit order	بيجيب الطلب ويملا الـ Dropdown	`View(order)` أو 404
`Edit(int id, Order)`	POST	`/Orders/Edit/5`	Save	زي Load-Copy-Save العملاء (العميل والتاريخ بس)	Redirect، أو الفورم بالأخطاء

Action	Method	الرابط	امتى بيتنده؟	بيعمل إيه؟	النتيجة
`Delete(int? id)`	GET	`/Orders/Delete/5`	Delete order	بيجيب الطلب مع العميل والعناصر عشان يعرض اللي هيتمسح	`View(order)` أو 404
`DeleteConfirmed(int id)`	POST	`/Orders/Delete/5`	Yes," "delete	بيمسح الطلب، وقاعدة البيانات بتمسح عناصره (Cascade)	Redirect للقائمة
`CheckOrderAsync(Order)`	private	—	من `Create` و `Edit` (POST)	الفحوصات اللي الـ Attributes ما تقدرش تعملها: التاريخ مش في المستقبل، والعميل المختار موجود فعلاً في قاعدة البيانات	أخطاء في `ModelState`
`FillCustomerListAsync()`	private	—	قبل عرض أي فورم طلب	بيجيب `Id` و `Name` بس لكل العملاء، ويحطهم في `ViewBag.Customers` كـ `SelectList`	—

5.5 `OrderItemsController`

مفيهوش `Index` ولا `Details` : العناصر بتظهر في صفحة تفاصيل الطلب. وبعد أي عملية، بيرجّعك لـ `/Orders/Details/{orderId}` .

Action	Method	الرابط	امتى بيتنده؟	بيعمل إيه؟
`Create(int? orderId)`	GET	`/OrderItems/Create?orderId=5`	"Add item" في صفحة الطلب	بيتأكد إن الطلب موجود، ويعرض فورم فيه `OrderId` في حقل مخفي
`Create(OrderItem)`	POST	`/OrderItems/Create`	Save	بيتأكد إن الطلب موجود (لأن الحقل المخفي ممكن يتلعب فيه)، الـ Attributes بتتحقق من الحقول، يحفظ، ويرجع للطلب
`Edit(int? id)`	GET	`/OrderItems/Edit/5`	أيقونة القلم جنب العنصر	بيعرض الفورم
`Edit(int id, OrderItem)`	POST	`/OrderItems/Edit/5`	Save	بياخد `OrderId` من قاعدة البيانات **مش من الفورم**، ينسخ القيم، ويحفظ
`Delete(int? id)`	GET	`/OrderItems/Delete/5`	أيقونة الزبالة جنب العنصر	صفحة "متأكد؟"
`DeleteConfirmed(int id)`	POST	`/OrderItems/Delete/5`	Yes," "delete	بيمسح العنصر ويرجع للطلب (حتى لو حصل خطأ، يرجع للطلب برسالة خطأ)

5.6 `PerformanceController`

Action	Method	الرابط	امتى بيتنده؟	بيعمل إيه؟
`Index()`	GET	`/Performance`	Performance في الـ Sidebar	بيعد صفوف الجداول ويقرا آخر 30 نتيجة من CSV
`Run(PerformanceViewModel)`	POST	`/Performance/Run`	زرار "Run test"	بيشغّل الـ 3 طرق، يتأكد إنهم رجّعوا نفس البيانات، يحفظ في CSV، ويعرض النتايج. بيرجّع `View("Index", model)`، فالرابط في المتصفح بيفضل `/Performance/Run`
`Generate(PerformanceViewModel)`	POST	`/Performance/Generate`	زرار "Delete all and generate" (بعد رسالة تأكيد في المتصفح)	يمسح كل البيانات بـ `RemoveRange`، ويعمل بيانات جديدة بـ `SeedData`، وبعدين Redirect لـ `/Performance`

والـ private methods فيه:

Method	وظيفتها
`MeasureAsync`	بتشغّل طريقة واحدة (تشغيلة تسخين + عدد التشغيلات المطلوب)، وبتمسح الـ Change Tracker قبل كل تشغيلة، وتقيس الوقت بـ `Stopwatch`. بتختار الطريقة بـ `if / else if / else`
`LoadWithNPlusOneAsync`	الطريقة 1: N+1 (تحميل صريح جوه حلقة)، وبتعدّ الاستعلامات بإيدها
`LoadWithIncludeAsync`	الطريقة 2: `Include` مع `AsNoTracking` (استعلام واحد)
`LoadWithSelectAsync`	الطريقة 3: Projection بـ `Select` (استعلام واحد)
`ToListItems`	بتحوّل `List<Order>` لـ `List<OrderListItem>`
`CountRowsAsync`	بتعد الصفوف في الـ 3 جداول
`SaveResults`	بتضيف سطور لملف CSV (ولو الملف مفتوح في Excel، بتعرض رسالة بدل ما الصفحة تقع)
`LoadHistory`	بتقرا آخر 30 سطر من CSV

6. شرح كل View ومكان ظهورها في المتصفح

6.1 مفاهيم

- **Razor:** لغة بتخلط HTML مع C#. أي حاجة بعد `@` بتتنفذ كـ C#، زي `@Model.Name` .
- **`@model` :** أول سطر في الـ View، بيحدد نوع البيانات اللي جاية من الـ Controller، زي `@model List<Customer>` .
- **Layout:** القالب الثابت (Sidebar وملفات CSS). كل View بتتحط جوه مكان `@RenderBody()` في الـ Layout.
- **Partial View:** View صغيرة بتتعاد في أكتر من صفحة، زي `_Pager` . بتتنده بـ `<partial name="_Pager" />` .
- **Tag Helper:** Attributes بتبدأ بـ `asp-` وبتتحول لـ HTML على السيرفر. مثلاً
  `asp-action="Details" asp-route-id="5"` بتتحول لـ `href="/Customers/Details/5"` .
- **`@* ... *@` :** تعليق Razor، مش بيوصل للمتصفح خالص.

6.2 كل View ومكانها

الـ View	بتظهر على	الـ Model	بتعرض إيه؟
`Home/Index.cshtml`	`/`	مفيش	عنوان Dashboard و 3 مربعات ملونة: Customers (أخضر)، Orders (بنفسجي)، Performance (غامق). كل مربع لينك
`Home/Privacy.cshtml`	`/Home/Privacy`	مفيش	نص مبدئي من القالب جوه كارت
`Home/HttpError.cshtml`	أي رابط رجّع 404 (الرابط في المتصفح بيفضل زي ما هو)	`ViewBag.Code`	لو 404: "Not found"، غير كده: "Something went wrong" مع الكود، وزراير للطلبات والعملاء
`Shared/Error.cshtml`	لما يحصل Exception في Production	`ErrorViewModel`	"Sorry, something went wrong" و `RequestId`
`Customers/Index.cshtml`	`/Customers`	`List<Customer>`	جدول: دايرة بأول حرف من الاسم، الإيميل، التليفون (أو `-`)، أيقونات عرض/تعديل/حذف، والـ Pager
`Customers/Details.cshtml`	`/Customers/Details/{id}`	`Customer` (مع `Orders`)	على الشمال: بيانات العميل وزراير Edit و Delete. على اليمين: جدول طلباته وزرار "New order for this customer"
`Customers/Create.cshtml`	`/Customers/Create`	`Customer`	فورم: Name و Email و Phone number
`Customers/Edit.cshtml`	`/Customers/Edit/{id}`	`Customer`	نفس الفورم بالبيانات الحالية
`Customers/Delete.cshtml`	`/Customers/Delete/{id}`	`Customer` + `ViewBag.OrderCount`	لو عنده طلبات: تحذير ومفيش زرار حذف. غير كده: "Are you sure?" وزرار "Yes, delete"

الـ View	بتظهر على	الـ Model	بتعرض إيه؟
`Orders/Index.cshtml`	`/Orders`	`List<OrderListItem>`	جدول: رقم الطلب وتحته التاريخ، العميل، عدد العناصر ("item 1" أو "items 3")، الإجمالي، والأيقونات، والـ Pager
`Orders/Details.cshtml`	`/Orders/Details/{id}`	`Order` (مع `Customer` و `Items`)	عنوان الطلب والتاريخ الطويل، كارت العميل (اسمه لينك لصفحته)، وجدول العناصر وإجمالي كل سطر وإجمالي الطلب
`Orders/Create.cshtml`	`/Orders/Create`	`Order` + `ViewBag.Customers`	Dropdown للعميل (أول اختيار "-- Choose a customer --" بقيمة 0) وحقل تاريخ (`type="date"`)
`Orders/Edit.cshtml`	`/Orders/Edit/{id}`	`Order` + `ViewBag.Customers`	نفس الفورم بالبيانات الحالية
`Orders/Delete.cshtml`	`/Orders/Delete/{id}`	`Order` (مع `Customer` و `Items`)	التاريخ والعميل وعدد العناصر، وتحذير إن العناصر هتتمسح معاه
`OrderItems/Create.cshtml`	`/OrderItems/Create?orderId={id}`	`OrderItem`	فورم: Product، و Quantity و Unit price جنب بعض، و `OrderId` في حقل مخفي
`OrderItems/Edit.cshtml`	`/OrderItems/Edit/{id}`	`OrderItem`	نفس الفورم (من غير حقل مخفي؛ الـ `id` في الرابط)
`OrderItems/Delete.cshtml`	`/OrderItems/Delete/{id}`	`OrderItem`	المنتج والكمية والسعر و "Yes, delete"
`Performance/Index.cshtml`	`/Performance` و `/Performance/Run`	`PerformanceViewModel`	3 مربعات بعدد الصفوف، فورم التشغيل، فورم إنشاء البيانات، جدول نتايج الطرق التلاتة بشرائط، الـ SQL بتاع كل طريقة، الشرح، والنتايج المحفوظة

6.3 الـ Layout بالتفصيل (`Views/Shared/_Layout.cshtml`)

ده الملف اللي بيدّي كل الصفحات نفس الشكل:

1. في أوله، بيعرف اسم الـ Controller الحالي عشان ينوّر اللينك الصح في القائمة (الكود تحت).
2. `<html lang="en" data-bs-theme="dark">` : الـ Attribute ده بيشغّل الوضع الغامق بتاع Bootstrap.
3. بيحمّل خط Inter من Google Fonts، وأيقونات Bootstrap Icons، و Bootstrap، وبعدين `site.css` .
4. الـ Sidebar: اسم الموقع، والقائمة (Dashboard و Customers و Orders و Performance)، ولينك Privacy تحت.
5. اللينك بياخد كلاس `active` لو اسم الـ Controller بتاعه هو الحالي. وصفحات `OrderItems` بتنوّر "Orders" (الكود تحت).
6. بيعرض `TempData["Success"]` في رسالة خضرا، و `TempData["Error"]` في رسالة حمرا.
7. `@RenderBody()` : هنا محتوى الصفحة الحالية بيتحط.

8. في الآخر: jQuery و Bootstrap JS و `site.js` ، و `@await RenderSectionAsync("Scripts", required: false)` اللي بيسمح لأي صفحة تضيف سكريبتات (كل الفورمز وصفحة الأداء بيستخدموه عشان يضيفوا سكريبتات التحقق).

الكود اللي بيعرف الصفحة الحالية (أول الملف):

```
string currentController = ViewContext.RouteData.Values["controller"]?.ToString() ?? "";
```

ولينك Orders في القائمة:

```
<a class="menu-link @(currentController == "Orders" || currentController == "OrderItems" ?
"active" : "")" ...>
```

6.4 الـ Pager (`Views/Shared/_Pager.cshtml`)

بياخد `ViewBag.Page` و `ViewBag.TotalPages` من الـ Controller، وما بيظهرش لو فيه صفحة واحدة بس. الزرايز مكتوبة كده:

```
<a class="page-link" asp-route-page="@(currentPage + 1)">Next <i class="bi bi-chevron-right"></i>
</a>
```

مفيش `asp-controller` ولا `asp-action` ، فالـ Tag Helper بيستخدم الصفحة الحالية ويضيف `?page=3` . عشان كده نفس الـ Partial شغال في العملاء والطلبات.

7. Routing والتنقل بين الصفحات

7.1 يعني إيه Routing؟

Routing هو اللي بيقرر: الرابط ده يروح لأنهي Controller وأنهي Action؟ في المشروع ده فيه Route واحد بس، متعرّف في آخر `Program.cs` (سطور 49 لـ 52):

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();
```

معنى الـ pattern:

- أول جزء من الرابط = اسم الـ Controller (لو مش موجود، يبقى `Home`).
- تاني جزء = اسم الـ Action (لو مش موجود، يبقى `Index`).
- تالت جزء = `id`، وعلامة `?` معناها إنه اختياري.

ده اسمه **Conventional Routing** (Routing بالاتفاق على شكل ثابت). المشروع **مش** بيستخدم `[Route]` فوق الـ Actions (Attribute Routing)، ومفيهوش Areas.

7.2 أمثلة من المشروع

الرابط	Controller	Action	البيانات اللي بتوصل
`/`	Home	Index	—
`/Customers`	Customers	Index	`page = 1` (القيمة الافتراضية)
`/Customers?page=3`	Customers	Index	`page = 3` من الـ Query String
`/Customers/Details/7`	Customers	Details	`id = 7`
`/Customers/Details/abc`	Customers	Details	`id = null` (لأن "abc" مش رقم)، فالـ Action بترجع 404
`/Orders/Create?customerId=5`	Orders	Create	`customerId = 5`
`/OrderItems/Create?orderId=12`	OrderItems	Create	`orderId = 12`
`/Performance`	Performance	Index	—

Query String: الجزء اللي بعد `?` في الرابط، زي `?page=3`. ASP.NET Core بيحط قيمته تلقائي في الـ Parameter اللي بنفس الاسم. والعملية دي اسمها **Model Binding**.

7.3 إزاي الروابط بتتعمل في الـ Views؟

ما فيش رابط مكتوب بإيد (زي `href="/Customers/Details/5"`). كل الروابط بتتعمل بالـ Tag Helpers:

```
<a asp-action="Details" asp-route-id="@customer.Id">...</a>
<a asp-controller="Orders" asp-action="Create" asp-route-customerId="@Model.Id">...</a>
```

الفايدة: لو غيّرت شكل الـ Route في `Program.cs`، كل الروابط بتتغير لوحدها. أي `asp-route-xxx` بيبقى جزء من الرابط لو اسمه موجود في الـ pattern (زي `id`)، وإلا بيتحط في الـ Query String (زي `customerId`).

7.4 GET و POST على نفس الرابط

صفحة الإضافة ليها Action اسمها `Create` مرتين:

- **GET** `/Customers/Create` : يعرض الفورم الفاضي.
- **POST** `/Customers/Create` : يستقبل البيانات (عليها `[HttpPost]`).

الحذف نفس الفكرة، بس الـ method التانية اسمها `DeleteConfirmed` عشان C# ما يسمحش بـ methodين بنفس الاسم ونفس الـ Parameters (`int id`). فبنستخدم `[ActionName("Delete")]` عشان ترد على نفس الرابط `/Customers/Delete/5`.

7.5 الانتقال بعد العمليات (Redirect)

بعد أي حفظ ناجح، الـ Controller مش بيعرض View مباشرة. بيرجّع `RedirectToAction`:

```
return RedirectToAction(nameof(Details), new { id = customer.Id });
```

السيرفر بيرد بكود **302** ومعاه رابط جديد (`/Customers/Details/123`)، والمتصفح بيروح عليه بـ GET. ولو الـ Redirect لـ Controller تاني:

```
return RedirectToAction("Details", "Orders", new { id = item.OrderId });
```

`nameof(Details)` بترجع النص `"Details"`، بس لو غيّرت اسم الـ method، الكومبايلر هيطلّعلك خطأ بدل ما الرابط يبوظ في السكوت.

7.6 روابط الأخطاء

لما أي Action ترجع `NotFound()`، الـ Middleware ده (سطر 39 في `Program.cs`):

```
app.UseStatusCodePagesWithReExecute("/Home/HttpError", "?code={0}");
```

بيعيد تشغيل الـ Pipeline **جوه السيرفر** على `/Home/HttpError?code=404`. المتصفح بيفضل شايف الرابط الأصلي، والكود بيفضل 404، بس المحتوى بيبقى صفحة "Not found" الحلوة بدل صفحة بيضا.

7.7 خريطة التنقل في الموقع

```
Sidebar (on every page): Dashboard | Customers | Orders | Performance | Privacy

Dashboard (/)
  ├── Customers → /Customers ──→ Details ──→ Edit / Delete
  │                    │              └──→ "New order for this customer" → /Orders/Create?
customerId=..
  │                    └── New customer → /Customers/Create ──(Save)──→ Details
  │
  ├── Orders ────→ /Orders ─────→ Details ──→ Edit order / Delete order
  │                    │              ├──→ customer name → /Customers/Details/..
  │                    │              └──→ Add item → /OrderItems/Create?orderId=..
  │                    │                    Edit/Delete item → /OrderItems/Edit|Delete/..
  │                    └── New order → /Orders/Create ──(Save)──→ Details (to add items)
  │
  └── Performance → /Performance ──(Run test)──→ /Performance/Run
                                  └(Generate)──→ /Performance (after a redirect)
```

8. EF Core و DbContext و Migrations والاتصال بقاعدة البيانات

8.1 يعني إيه EF Core؟

Entity Framework Core هو **ORM** (Object-Relational Mapper): مكتبة بتحوّل الكلاسات لجداول، والكائنات لصفوف، واستعلامات LINQ في C# لاستعلامات SQL. يعني بدل ما تكتب:

```
SELECT * FROM Customers WHERE Id = 5
```

بتكتب:

```
await _db.Customers.FirstOrDefaultAsync(c => c.Id == 5);
```

LINQ (Language Integrated Query): طريقة بتكتب بيها استعلامات جوه C# باستخدام methods زي `Where` و `OrderBy` و `Select`.

8.2 الاتصال بقاعدة البيانات

1. الـ Connection String في `appsettings.json`:

```
"ConnectionStrings": {
  "DefaultConnection":
"Server=.\\SQLEXPRESS;Database=SalesOrdersDb;Trusted_Connection=True;TrustServerCertificate=True;"
}
```

الجزء	معناه
`Server=.\SQLEXPRESS`	النقطة `.` يعني نفس الجهاز، و `SQLEXPRESS` اسم الـ Instance (نسخة SQL Server Express). في JSON الـ `\` بتتكتب `\\`
`Database=SalesOrdersDb`	اسم قاعدة البيانات
`Trusted_Connection=True`	الدخول بحساب Windows الحالي (Windows Authentication)، فمفيش User أو Password في الملف
`TrustServerCertificate=True`	اقبل شهادة التشفير بتاعة السيرفر من غير تحقق. ده مناسب للتطوير على جهازك، **مش** لسيرفر حقيقي

2. تسجيل الـ DbContext في `Program.cs` (سطور 16 و 18):

```
string connectionString = builder.Configuration.GetConnectionString("DefaultConnection")!;
builder.Services.AddDbContext<AppDbContext>(options => options.UseSqlServer(connectionString));
```

- `GetConnectionString("DefaultConnection")` بيقرا القيمة من `ConnectionStrings` في `appsettings.json`.
- علامة `!` في الآخر بتقول للكومبايلر "متأكد إنها مش null" (اسمها **null-forgiving operator**).
- `AddDbContext` بيسجّل `AppDbContext` في الـ DI بـ Scoped lifetime: نسخة جديدة لكل Request.
- `UseSqlServer` بيقول لـ EF يستخدم الـ Provider بتاع SQL Server.

8.3 الـ DbContext (`Data/AppDbContext.cs`)

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Customer> Customers { get; set; } = default!;
    public DbSet<Order> Orders { get; set; } = default!;
    public DbSet<OrderItem> OrderItems { get; set; } = default!;

    protected override void OnModelCreating(ModelBuilder modelBuilder) { /* Fluent API */ }
}
```

- **`DbContext`:** الكلاس الأساسي في EF. بيمثل "جلسة شغل" مع قاعدة البيانات: بيعمل استعلامات، وبيتابع التغييرات، وبيحفظها.
- **Constructor:** بياخد الـ options (Provider + Connection String) من `Program.cs` وبيبعتها للـ base class.
- **`DbSet<T>`:** بيمثل جدول. `_db.Customers` هو جدول العملاء، وتقدر تعمل عليه استعلامات. و `= default!` معناها: "يا كومبايلر ما تقلقش، EF Core هو اللي هيملا الـ Property دي". من غيرها الكومبايلر بيطلّع تحذير، لأن المشروع مفعّل Nullable Reference Types. ودي نفس الطريقة اللي Visual Studio بيكتبها لما يعمل Scaffolding.

8.4 الـ Fluent API

Fluent API: طريقة بتعرّف بيها قواعد الجداول بالكود، عن طريق methods ورا بعض (عشان كده اسمها Fluent: "سلسة"). كل الـ Methods المستخدمة في المشروع:

Method	معناها	مثال من المشروع
`ToTable("...")`	اسم الجدول	`entity.ToTable("Customers")`
`HasKey(...)`	الـ Primary Key	`entity.HasKey(c => c.Id)`
`Property(...)`	اختيار عمود عشان نحط قواعده	`entity.Property(c => c.Name)`
`IsRequired()`	`NOT NULL`	`.IsRequired()` على Name و Email و OrderDate و ProductName و Quantity و UnitPrice
`IsRequired(false)`	مسموح NULL	على Phone
`HasMaxLength(n)`	`nvarchar(n)`	100 لـ Name و 200، 30 لـ ProductName، Email لـ Phone
`HasPrecision(18, 2)`	`decimal(18,2)`	على UnitPrice
`HasIndex(...)`	عمل Index	على Name و Email و OrderDate
`IsUnique()`	الـ Index يمنع التكرار	على Email
`HasOne / WithMany / HasForeignKey`	تعريف علاقة One-to-Many	Customer–Orders و Order–Items
`OnDelete(...)`	اللي يحصل عند الحذف	`Restrict` و `Cascade`

8.5 الـ Migrations

Migration: ملف C# بيوصف تغيير في شكل قاعدة البيانات (عمل جدول، إضافة عمود، ...). بدل ما تعدّل قاعدة البيانات بإيدك، بتغيّر الـ Models، وتعمل Migration، وتطبّقها. وكده كل تغييرات قاعدة البيانات بتبقى متسجلة في الكود.

الأوامر:

الهدف	من Package Manager Console	من الـ Terminal
عمل Migration جديدة	`Add-Migration InitialCreate`	`dotnet ef migrations add InitialCreate`
تطبيقها على قاعدة البيانات	`Update-Database`	`dotnet ef database update`
التراجع عن آخر Migration (لو ما اتطبقتش)	`Remove-Migration`	`dotnet ef migrations remove`
أعرف هل الكود اتغير بعد آخر Migration؟	—	`dotnet ef migrations has-pending-model-changes`

اللي حصل في المشروع:

1. اتعملت Migration واحدة اسمها `InitialCreate`، عملت الـ 3 جداول والـ 5 Indexes.
2. اتطبقت بـ `Update-Database` على `.\SQLEXPRESS`، فاتعملت قاعدة `SalesOrdersDb`.
3. لما نقلنا القواعد من Attributes لـ Fluent API، شغّلنا `has-pending-model-changes` وطلع "No changes have been made to the model since the last migration". يعني شكل قاعدة البيانات ما اتغيرش، فما احتجناش Migration جديدة.
4. ولما رجّعنا Attributes التحقق (`[Required]` و `[StringLength]` ...) فوق الـ Properties، شغّلنا نفس الأمر تاني، وبرضه طلع "No changes"، لأن الأطوال والإجباري زي ما هم في الـ Fluent API بالظبط.

لو عايز تضيف عمود جديد (مثلاً `Address` للعميل):

1. ضيف `public string? Address { get; set; }` في `Customer`.
2. ضيف قواعده في `OnModelCreating` لو محتاج (مثلاً `HasMaxLength(250)`).
3. `Add-Migration AddCustomerAddress`.
4. `Update-Database`.

8.6 البيانات التجريبية (Seeding)

في `Program.cs` (سطور 22 لـ 30)، بعد `builder.Build()` وقبل ما التطبيق يبدأ:

```
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    if (!db.Customers.Any())
    {
        SeedData.AddTestData(db, 100);
    }
}
```

- `CreateScope()` : محتاجينها لأن `AppDbContext` متسجل Scoped، ومفيش Request دلوقتي، فبنعمل Scope بإيدينا.
- `GetRequiredService<AppDbContext>()` : بنطلب DbContext من الـ DI.
- لو مفيش ولا عميل، بنضيف 100 عميل ببياناتهم.

`SeedData.AddTestData` بيعمل:

- `new Random(12345)` : رقم ثابت (Seed)، فالأرقام "العشوائية" بتطلع **نفس الأرقام كل مرة**. ده بيخلي اختبارات الأداء ممكن تتكرر على نفس البيانات.
- كل عميل: الاسم `"Customer " + c` ، والإيميل `"customer" + c + "@example.com"` ، والتليفون `"555-" + c.ToString("0000")` .
- كل عميل عنده من 1 لـ 10 طلبات، بتواريخ عشوائية في سنة 2025.
- كل طلب فيه من 1 لـ 5 عناصر، من 10 منتجات ثابتة، بكمية من 1 لـ 5 وسعر من 1.00 لـ 499.99.
- في الآخر `db.SaveChanges()` مرة واحدة بس.

> **مهم:** الكود ده محتاج الجداول تكون موجودة. لو شغّلت المشروع على قاعدة بيانات جديدة من غير `Update-Database` ، هيطلع Exception عند `db.Customers.Any()` .

8.7 الاستعلامات: من LINQ لـ SQL

```
List<Customer> customers = await _db.Customers
    .AsNoTracking()
    .OrderBy(c => c.Name)
    .Skip((page - 1) * PageSize)
    .Take(PageSize)
    .ToListAsync();
```

- كل method قبل `ToListAsync` بتبني الاستعلام بس، ومش بتكلم قاعدة البيانات (ده اسمه **Deferred Execution**، والنوع اسمه `IQueryable`).
- `ToListAsync()` هي اللي بتبعت الاستعلام فعلاً وبترجع النتايج.
- `Skip` و `Take` بيتحولوا لـ `OFFSET ... FETCH` في SQL Server، فقاعدة البيانات بترجع 20 صف بس، مش كل العملاء.

Methods بتنفذ الاستعلام فوراً: `ToListAsync` و `FirstOrDefaultAsync` و `CountAsync` و `AnyAsync` و `FindAsync` .

`FindAsync(id)` : بيدور بالـ Primary Key. ميزته إنه بيدور الأول في الكائنات اللي الـ DbContext متابعها، ولو ما لقاش، بيسأل قاعدة البيانات.

8.8 Tracking و AsNoTracking

Change Tracker: جزء في الـ DbContext بيحفظ نسخة من كل كائن اتحمّل، عشان لما تغيّر فيه وتعمل `SaveChangesAsync` ، يعرف إيه اللي اتغير.

- **مع Tracking (الافتراضي):** مناسب لما هتعدّل البيانات (زي `FindAsync` في Edit).
- **`AsNoTracking()` :** مناسب للصفحات اللي بتعرض بس. أسرع وبياكل ذاكرة أقل، لأن EF مش بيحفظ نسخ.

في المشروع: كل صفحات العرض (`Index` و `Details` و صفحة `Delete` GET) بتستخدم `AsNoTracking` ، وكل صفحات التعديل بتستخدم `FindAsync` (Tracking).

ملحوظة: الـ Projection (`Select` لـ `OrderListItem`) مش محتاج `AsNoTracking` ، لأن EF مش بيتابع غير الـ Entities.

8.9 تحميل البيانات المرتبطة (أهم جزء في الـ Interview)

الطريقة	الكود	عدد الاستعلامات	مستخدمة فين في المشروع
Eager Loading	`.Include(o => o.Customer).Include(o => o.Items)`	1 (بـ JOIN)	`OrdersController.Details` و `Delete` ، والطريقة 2 في صفحة الأداء
Filtered Include	`.Include(c => c.Orders.OrderByDescending(o => o.OrderDate))`	1	`CustomersController.Details` (طلبات العميل مترتبة)
Split Query	`Include` + `.AsSplitQuery()`	2	**مش مستخدمة** (مفيدة لما الطلب يبقى فيه عناصر كتير جداً)
Explicit Loading	`db.Entry(order).Collection(o => o.Items).LoadAsync()`	1 لكل عنصر	الطريقة 1 (N+1) في صفحة الأداء بس، كمثال على الطريقة الغلط للقوائم
Projection	`.Select(o => new OrderListItem { ... })`	1	`OrdersController.Index` ، والطريقة 3 في صفحة الأداء
Lazy Loading	—	—	**مش مستخدمة**. محتاجة package اسمه `Proxies` و `virtual` على الـ Navigation Properties. وبتعمل نفس مشكلة N+1 من غير ما تاخد بالك

مشكلة N+1: بتجيب N طلب في استعلام واحد، وبعدين بتعمل استعلام لكل طلب عشان تجيب عناصره. يعني 1 + N استعلام. لو عندك 1000 طلب، يبقى 1001 رحلة لقاعدة البيانات.

أرقام حقيقية من ملف `perf-results.csv` (اختبار يوم 2026-10-21 الساعة 02:08، على 100 عميل و 556 طلب و 1691 عنصر، تحميل 200 طلب، 5 تشغيلات). الاختبار ده اتعمل وصفحة الأداء كان فيها 5 طرق؛ دي أرقام الطرق التلاتة اللي فاضلة في الصفحة دلوقتي:

الطريقة (في الصفحة دلوقتي)	الاسم في ملف CSV	متوسط الوقت	عدد الاستعلامات
N+1 (load in a loop)	N+1 (explicit loading in a loop)	431.05 ms	281
Include (eager loading)	Include + AsNoTracking	27.90 ms	1
Select (projection)	Projection (Select)	6.14 ms	1

ليه N+1 عمل 281 استعلام مش 401؟ المفروض 1 + (200 × 2) = 401. بس لما EF يحمّل عميل مرة، الطلبات التانية بتاعة نفس العميل بيبقى عميلها متحمّل خلاص (`IsLoaded = true`)، فالكود مش بيسأل عنه تاني. فالعدد = 1 للطلبات + 200 للعناصر + 80 عميل مختلف. واتأكدت من رقم 80 ده باستعلام SQL مباشر على نفس البيانات، والصفحة طلّعت 281 بالظبط.

الخلاصة اللي المشروع وصل لها:

- **صفحات القوائم:** Projection (أسرع وأقل بيانات)، وده المستخدم في `/Orders` .

- **صفحات التفاصيل:** `Include` + `AsNoTracking` (محتاجين الكائنات كاملة)، وده المستخدم في `/Orders/Details` .
- **N+1:** ممنوعة في القوائم.

> **ملحوظة:** الأرقام دي من جهازك وقت الاختبار ده. الأرقام بتختلف من جهاز لجهاز ومن تشغيلة للتانية، بس الترتيب عادةً بيفضل زي ما هو.

8.10 الحفظ: `SaveChangesAsync`

```
_db.Customers.Add(customer);   // الكائن بيتعلّم إنه جديد، ولسه ما اتحفظش
await _db.SaveChangesAsync();  // هنا بس أمر الإضافة بيتبعت لقاعدة البيانات
```

- `Add` و `Remove` وتغيير Properties كائن متتبَّع: كلهم بيتسجلوا في الـ Change Tracker بس.
- `SaveChangesAsync` بتحوّل كل التغييرات لـ `INSERT` و `UPDATE` و `DELETE` ، وبتنفذهم جوه **Transaction** واحدة تلقائي (يا كلهم ينجحوا، يا مفيش ولا واحد).
- بعد الـ `INSERT` ، EF بيحط الـ `Id` اللي SQL Server ولّده جوه الكائن. عشان كده `customer.Id` بيبقى فيه قيمة في السطر اللي بعده (`RedirectToAction(nameof(Details), new { id = customer.Id })`).

8.11 حاجات EF مستخدمة في صفحة الأداء

الحاجة	فين	بتعمل إيه
`ChangeTracker.Clear()`	`MeasureAsync`	بتخلّي الـ DbContext ينسى كل الكائنات اللي اتحمّلت قبل كده، عشان كل تشغيلة تبدأ من نفس الظروف
`ToQueryString()`	الطرق التلاتة	بترجّع نص الـ SQL اللي EF هيبعته، من غير ما تنفذه. بنعرضه في الصفحة
`Entry(order).Reference(...).IsLoaded`	طريقة N+1	بتقول هل العميل اتحمّل خلاص ولا لأ، عشان نعدّ الاستعلامات صح
`RemoveRange(...)`	`Generate`	بيعلّم كل الصفوف إنها هتتمسح، و `SaveChangesAsync` بيبعت أوامر `DELETE`

8.12 تسجيل استعلامات SQL (Logging)

في `appsettings.json` :

```
"Microsoft.EntityFrameworkCore.Database.Command": "Warning"
```

ده معناه إن EF **مش** بيطبع كل استعلام SQL في الـ Console. اتعمل كده لأن الطباعة كانت بتبطّأ إنشاء آلاف الصفوف. لو عايز تشوف كل SQL في الـ Console، غيّرها لـ `"Information"` . وكبديل، صفحة الأداء بتعرض الـ SQL الحقيقي لكل طريقة (من `ToQueryString()`).

9. عمليات CRUD: رحلة البيانات من المستخدم لقاعدة البيانات والعكس

CRUD = Create (إضافة)، Read (قراءة)، Update (تعديل)، Delete (حذف).

العملية	العملاء	الطلبات	العناصر	SQL في الآخر
Create	GET + POST `Create`	GET + POST `Create`	GET + POST `Create`	`INSERT`
Read	`Index` ، `Details`	`Index` ، `Details`	جوه `Orders/Details`	`SELECT`
Update	GET + POST `Edit`	GET + POST `Edit`	GET + POST `Edit`	`UPDATE`
Delete	GET + `DeleteConfirmed` POST `Delete`	نفس الشكل	نفس الشكل	`DELETE`

9.1 الرحلة العامة لأي فورم

مثال: إضافة عميل.

1. `GET /Customers/Create` : الـ Controller يرجّع View فيها فورم فاضي.
2. المستخدم بيملا الفورم ويدوس Save.
3. `POST /Customers/Create` : المتصفح بيبعت `Name` و `Email` و `Phone` و `__RequestVerificationToken` .
4. `[ValidateAntiForgeryToken]` : التأكد إن الفورم جاي من موقعنا.
5. ASP.NET Core :**Model Binding** بيعمل كائن `Customer` ويملا `Name` و `Email` و `Phone` .
6. **Validation**: الـ Attributes اللي في `Customer` (زي `[Required]`) بتتحقق تلقائي، وبعدين الـ Controller بيتأكد إن الإيميل مش متكرر. أي خطأ بيتحط في `ModelState` . (ولو JavaScript شغال، المتصفح بيعمل نفس فحوصات الـ Attributes قبل ما الفورم يتبعت أصلاً.)
7. لو فيه أخطاء: `return View(customer)` ، يعني نفس الفورم بالقيم اللي اتكتبت والأخطاء، ومفيش أي حفظ.
8. لو مفيش أخطاء: `_db.Customers.Add(customer)` وبعدين `SaveChangesAsync()` ، فبيتبعت `INSERT` لـ SQL Server.
9. `TempData["Success"] = "..."` : رسالة للصفحة الجاية.
10. `RedirectToAction(nameof(Details), ...)` : السيرفر بيرد بكود 302.
11. `GET /Customers/Details/123` : استعلام SELECT، والصفحة بتظهر ومعاها الرسالة الخضرا.

9.2 Model Binding: البيانات بتوصل للـ Controller إزاي؟

في الـ View:

```
<input asp-for="Email" class="form-control" />
```

بيتحول لـ:

```
<input class="form-control" type="text" id="Email" name="Email" value="">
```

لما الفورم يتبعت، `name="Email"` بيروح لـ `customer.Email` في الـ Action، لأن الاسم زي اسم الـ Property. والـ Form Tag Helper (`<form asp-action="Create" method="post">`) بيضيف لوحده حقل مخفي اسمه `__RequestVerificationToken` للحماية من CSRF.

ملحوظة: لو الحقل فاضي، Model Binding بيحط `null` بدل `""` (ده الافتراضي في ASP.NET Core). عشان كده التحقق بيستخدم `string.IsNullOrWhiteSpace(...)` اللي بيمسك الحالتين.

9.3 نمط Post/Redirect/Get (PRG)

بعد POST ناجح، بنعمل Redirect بدل ما نعرض View على طول. **ليه؟** لو عرضنا View بعد POST والمستخدم داس Refresh، المتصفح هيبعت الـ POST تاني، فالعميل هيتضاف مرتين. مع الـ Redirect، الـ Refresh بيعيد الـ GET الأخير بس.

> **استثناء:** `PerformanceController.Run` بيرجّع `View("Index", model)` بعد POST من غير Redirect (عشان النتايج كبيرة ومش محتاجين نحفظها في TempData). فلو عملت Refresh على `/Performance/Run`، المتصفح هيسألك تعيد الإرسال، والاختبار هيتشغل تاني.

9.4 Create: الكود الحقيقي (`CustomersController`، سطور 78 لـ 113)

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create([Bind("Name,Email,Phone")] Customer customer)
{
    if (ModelState.IsValid)
    {
        bool emailTaken = await _db.Customers.AnyAsync(c => c.Email == customer.Email);
        if (emailTaken)
        {
            ModelState.AddModelError("Email", "Another customer already uses this email.");
        }
    }

    if (!ModelState.IsValid)
    {
        return View(customer);
    }

    try
    {
        _db.Customers.Add(customer);
        await _db.SaveChangesAsync();
        TempData["Success"] = "The customer was created.";
        return RedirectToAction(nameof(Details), new { id = customer.Id });
    }
    catch (DbUpdateException ex)
    {
        _logger.LogError(ex, "Could not create a customer.");
        ModelState.AddModelError("", "The customer could not be saved. Please try again.");
        return View(customer);
    }
}
```

9.5 Read: القائمة والتفاصيل

- **القائمة** (`OrdersController.Index`): `CountAsync` عشان عدد الصفحات، وبعدين Projection لـ 20 صف. يعني **استعلامين** في كل صفحة، مهما كان عدد الطلبات.

- **التفاصيل** (`OrdersController.Details`): استعلام واحد بـ `Include` للعميل والعناصر.
- **إجمالي الطلب في صفحة التفاصيل** بيتحسب في الـ View نفسها (في الذاكرة، لأن العناصر اتحمّلت خلاص). الكود تحت.

```
decimal orderTotal = Model.Items.Sum(i => i.Quantity * i.UnitPrice);
```

9.6 Update: طريقة Load-Copy-Save (`CustomersController` ، سطور 136 لـ 182)

```
Customer? savedCustomer = await _db.Customers.FindAsync(id);    // 1. هات النسخة المحفوظة
if (savedCustomer == null) return NotFound();                    //    لو مش موجود: صفحة 404
customer.Id = id;
// 2. اتأكد إن الإيميل مش عند عميل تاني
if (ModelState.IsValid && await _db.Customers.AnyAsync(c => c.Email == customer.Email && c.Id !=
id))
    ModelState.AddModelError("Email", "Another customer already uses this email.");
if (!ModelState.IsValid) return View(customer);
savedCustomer.Name = customer.Name;                              // 3. انسخ القيم الجديدة
savedCustomer.Email = customer.Email;
savedCustomer.Phone = customer.Phone;
await _db.SaveChangesAsync();                                    // 4. احفظ الأعمدة اللي اتغيرت بس
```

(نسخة مختصرة؛ الكود الحقيقي فيه `try/catch` حوالين الحفظ.)

ليه مش `_db.Update(customer)` على طول؟

1. **الأمان:** بننسخ بس الحقول المسموح بيها. في `OrderItemsController.Edit` ، `OrderId` بيتاخد من قاعدة البيانات، فمحدش يقدر ينقل عنصر لطلب تاني بالتلاعب في الفورم.
2. **التأكد من الوجود:** لو السجل اتمسح، بنرجع 404 بوضوح.
3. **الكفاءة:** EF بيبعت `UPDATE` للأعمدة اللي اتغيرت بس.

9.7 Delete: خطوتين

1. **GET** `/Customers/Delete/5` : صفحة تأكيد. الـ GET **عمره ما بيمسح** حاجة، لأن الـ GET لازم يبقى آمن (المتصفحات ومحركات البحث ممكن تفتح الروابط لوحدها).
2. **POST** `/Customers/Delete/5` : الحذف الفعلي، ومحمي بـ Anti-forgery Token.

```
bool hasOrders = await _db.Orders.AnyAsync(o => o.CustomerId == id);
if (hasOrders)
{
    TempData["Error"] = "This customer has orders, so it cannot be deleted. Delete the orders
first.";
    return RedirectToAction(nameof(Details), new { id });
}
_db.Customers.Remove(customer);
await _db.SaveChangesAsync();
```

ولما الطلب يتمسح، `_db.Orders.Remove(order)` بيبعت `DELETE` للطلب بس، وقاعدة البيانات هي اللي بتمسح العناصر بسبب `ON DELETE CASCADE` .

10. Validation و Error Handling و Authentication

10.1 Validation: التحقق من البيانات

Validation: التأكد إن البيانات اللي المستخدم دخّلها صحيحة قبل ما تتحفظ. التحقق في المشروع على **4 طبقات**:

الطبقة	فين؟	بتمسك إيه؟
1. في المتصفح	الـ Attributes اللي في الـ Models + مكتبة jQuery Validation	حقل فاضي، طول زيادة، إيميل أو تليفون شكله غلط، رقم برة النطاق. **قبل** ما الفورم يتبعت
2. على السيرفر	نفس الـ Attributes، و ASP.NET Core بيتحقق منها تلقائي قبل الـ Action	نفس الحاجات، حتى لو JavaScript مقفول في المتصفح
3. فحوصات زيادة في الـ Controllers	`if` عادية بتكلم قاعدة البيانات	إيميل متكرر، تاريخ في المستقبل، عميل مش موجود
4. قاعدة البيانات	قيود الجداول (NOT NULL، الأطوال، Unique، Foreign Keys)	آخر خط دفاع لو حاجة عدّت من كل اللي فات

10.2 الـ Attributes بتشتغل إزاي في المتصفح وعلى السيرفر؟

على السيرفر: قبل ما الـ Action تشتغل، ASP.NET Core بيعمل Model Binding، وبعدين بيتحقق من كل Attribute في الـ Model. أي خطأ بيتحط في `ModelState`، ووجوه الـ Action بنسأل `ModelState.IsValid`.

في المتصفح: الـ Tag Helper `asp-for` بيقرا الـ Attributes من الـ Model، ويحطها في الـ HTML على شكل `data-val-*`. مثلاً `[Required(ErrorMessage = "Please enter the product name.")]` بيطلع في الـ HTML كـ `data-val-required="Please enter the product name."`. ومكتبة **jQuery Validation Unobtrusive** بتقرا الحاجات دي وتتحقق قبل ما الفورم يتبعت. السكريبتات دي بتتضاف في آخر كل فورم:

```
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

يعني بنكتب القاعدة **مرة واحدة** في الـ Model، وهي بتشتغل في المكانين.

10.3 كل قواعد التحقق

العملاء:

الحقل	القاعدة	جاية منين؟	الرسالة
Name	إجباري	`[Required]`	Please enter the customer's name.
Name	≤ 100 حرف	`[StringLength(100)]`	The name can be at most 100 characters.
Email	إجباري	`[Required]`	Please enter an email address.
Email	شكل إيميل	`[EmailAddress]`	Please enter a valid email address.
Email	≤ 200 حرف	`[StringLength(200)]`	The email can be at most 200 characters.

الحقل	القاعدة	جاية منين؟	الرسالة
Email	مش مستخدم عند عميل **تاني**	`if` في `CustomersController`	Another customer already uses this email.
Phone	اختياري، بس لو موجود لازم شكله تليفون	`[Phone]`	Please enter a valid phone number.
Phone	≤ 30 حرف	`[StringLength(30)]`	The phone number can be at most 30 characters.

الطلبات:

الحقل	القاعدة	جاية منين؟	الرسالة
OrderDate	إجباري	`[Required]`	Please enter the order date.
OrderDate	مش في المستقبل	`CheckOrderAsync`	The order date cannot be in the future.
CustomerId	لازم يتختار عميل (مش 0)	`[Range(1, int.MaxValue)]`	Please choose a customer.
CustomerId	العميل موجود فعلاً في قاعدة البيانات	`CheckOrderAsync`	The chosen customer does not exist.

العناصر:

الحقل	القاعدة	جاية منين؟	الرسالة
ProductName	إجباري، و ≤ 100 حرف	`[Required]` و `[StringLength(100)]`	Please enter the product name.
Quantity	إجباري، ومن 1 لـ 1000	`[Required]` و `[Range(1, 1000)]`	Quantity must be between 1 and 1000.
UnitPrice	إجباري، ومن 0.01 لـ 100,000	`[Required]` و `[Range(0.01, 100000)]`	Unit price must be between 0.01 and 100,000.

ليه فيه فحوصات في الـ Controllers كمان؟ لأن الـ Attribute بيبص على قيمة الحقل لوحده، ومش بيقدر يسأل قاعدة البيانات. فـ "الإيميل مش متكرر" و "العميل موجود" محتاجين استعلام. و "التاريخ مش في المستقبل" محتاج يقارن بتاريخ النهارده. فدول متكتبين `if` عادية في الـ Controller.

رسايل MVC الافتراضية: لو حد كتب كلام في خانة رقم، المتصفح بيعرض رسالة MVC الجاهزة "The field Unit price must be a number.". ولو خانة رقم أو تاريخ وصلت للسيرفر فاضية (يعني JavaScript كان مقفول)، السيرفر بيعرض الرسالة الجاهزة "The value '' is invalid.". دول مش رسايل احنا كاتبينها.

10.4 الرسايل بتظهر فين في الصفحة؟

```
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
...
<input asp-for="Email" class="form-control" />
<span asp-validation-for="Email" class="text-danger"></span>
```

- `asp-validation-for="Email"` : بيعرض أول خطأ متسجل على حقل `Email` تحته على طول.

- `asp-validation-summary="ModelOnly"` : بيعرض الأخطاء اللي **مش** تابعة لحقل معين. مثلاً `ModelState.AddModelError("", "The customer could not be saved...")` : المفتاح `""` معناه "خطأ عام".

10.5 التحقق في المتصفح موجود في كل الفورمز

كل الفورمز (العملاء والطلبات والعناصر وصفحة الأداء) فيها سكريبتات التحقق. فلو المستخدم داس Save والفورم فيه غلط، الرسايل بتظهر على طول من غير ما الصفحة تروح للسيرفر وترجع. ولو JavaScript مقفول، السيرفر بيمسك نفس الأخطاء ويرجّع الفورم بالرسايل.

> **تاريخ صغير:** في نسخة قبل كده، الـ Attributes كانت متشالة من الـ Models والتحقق كان مكتوب بإيد في الـ Controllers، فالسكريبتات دي كانت متشالة. بعد تبسيط المشروع، الـ Attributes رجعت ومعاها التحقق في المتصفح (التفاصيل في القسم 14).

10.6 Error Handling: التعامل مع الأخطاء

الموقف	اللي بيحصل في الكود	اللي المستخدم بيشوفه
رابط من غير `id` ، أو `id` مش رقم	`if (id == null) return NotFound();`	صفحة "Not found" (404)
سجل مش موجود أو اتمسح	`if (customer == null) return NotFound();`	صفحة "Not found"
خطأ وقت الحفظ في قاعدة البيانات	`catch (DbUpdateException ex)` ← `_logger.LogError(...)` ← رسالة بسيطة	نفس الفورم ورسالة حمرا "could not be saved. Please try again."
حذف عميل عنده طلبات	فحص `AnyAsync` قبل الحذف ← `TempData["Error"]`	رجوع لصفحة العميل ورسالة حمرا
ملف CSV مفتوح في Excel	`catch (IOException ex)` ← `model.Message`	تحذير أصفر في صفحة الأداء، والصفحة بتشتغل عادي
فشل إنشاء البيانات التجريبية	`catch (DbUpdateException ex)` ← `TempData["Error"]`	"The test data could not be created. Please try again." (مفيش Transaction، فممكن تكون البيانات القديمة اتمسحت)
Exception مش متوقع **في Development**	ASP.NET Core بيعرض **Developer Exception Page** تلقائي	صفحة فيها تفاصيل الخطأ والـ Stack Trace (زي صورة الخطأ "The view 'Index' was not found" اللي ظهرت قبل كده)
Exception مش متوقع **في Production**	`app.UseExceptionHandler("/Home/Error")`	صفحة "Sorry, something went wrong" و `RequestId` بس، **من غير أي تفاصيل تقنية**

Logging: كل Controller بياخد `ILogger<T>` من الـ DI.

`_logger.LogError(ex, "Could not update customer {CustomerId}.", id)` بيكتب الخطأ الكامل في الـ Console و Output في Visual Studio. `{CustomerId}` اسمه **Structured Logging**: القيمة بتتحفظ كحقل منفصل، فيسهل البحث بيها بعدين. المستخدم عمره ما بيشوف الكلام ده.

Development و Production: البيئة بتتحدد بمتغير `ASPNETCORE_ENVIRONMENT` . في `launchSettings.json` القيمة `Development` ، فلما تشغّل من Visual Studio بتشوف التفاصيل. على سيرفر حقيقي من غير المتغير ده، البيئة الافتراضية بتبقى

.Production

10.7 حماية موجودة فعلاً

الحماية	فين
Anti-forgery Token ضد CSRF	`[ValidateAntiForgeryToken]` على كل POST، والـ Form Tag Helper بيضيف الـ Token لوحده
منع Overposting	`[Bind(...)]` + طريقة Load-Copy-Save + `OrderId` من قاعدة البيانات في تعديل العنصر
منع XSS	Razor بيعمل HTML Encoding تلقائي لأي `@...` . لو حد كتب `<script>` كاسم عميل، هيظهر كنص عادي
HTTPS	`UseHttpsRedirection()` ، و `UseHsts()` في Production بس
عدم كشف التفاصيل	رسايل بسيطة للمستخدم، والتفاصيل في الـ Log بس
عدم حفظ باسورد في الكود	`Trusted_Connection=True` (Windows Authentication)

10.8 Authentication و Authorization: غير موجودين

- **Authentication:** معرفة المستخدم مين (Login). **مش موجود.** مفيش صفحة Login، ولا ASP.NET Core Identity، ولا جدول Users.
- **Authorization:** المستخدم مسموح له يعمل إيه. **مش موجود فعلياً.** السطر `app.UseAuthorization();` في `Program.cs` جاي من القالب، بس مفيش أي `[Authorize]` على أي Controller، فأي حد يفتح الموقع يقدر يضيف ويعدّل ويمسح كل حاجة.

لو اتسألت هتضيفهم إزاي: ASP.NET Core Identity لتسجيل الدخول، و `[Authorize]` على الـ Controllers، و Roles (مثلاً Admin بس هو اللي يمسح أو يشغّل Generate).

11. تصميم الواجهة: CSS و JavaScript والمكتبات

11.1 الفكرة

تصميم **Dashboard غامق**: Sidebar على الشمال بأيقونات، وكروت رمادي غامق، ولون أساسي أخضر فاتح، ولون تاني بنفسجي، ودوائر بأول حرف من الاسم، و Labels صغيرة (Pills). الشكل مستوحى من صور Dashboards اتبعتت كمرجع، **من غير أي Feature جديدة**: نفس الصفحات ونفس البيانات.

11.2 المكتبات

المكتبة	النسخة	مصدرها	مستخدمة في إيه؟
Bootstrap (CSS)	5.3.3	محلي: `wwwroot/lib/bootstrap`	الـ Grid (`row` و `col-md-4`)، الجداول، الفورمز، الزرايز، الوضع الغامق
Bootstrap (JS)	5.3.3	محلي	بيتحمّل في الـ Layout، بس **مفيش** Component تفاعلي بيستخدمه دلوقتي (مفيش Modal ولا Dropdown)
Bootstrap Icons	1.11.3	CDN (`cdn.jsdelivr.net`)	كل الأيقونات، زي `<i class="bi bi-people"></i>`
خط Inter	—	Google Fonts	خط الموقع كله
jQuery	3.7.1	محلي	مطلوب لمكتبات التحقق
jQuery Validation + Unobtrusive	1.21.0 و 4.0.0	محلي	التحقق في المتصفح في كل الفورمز وصفحة الأداء

> **لازم نت:** الأيقونات والخط بيتحمّلوا من النت (CDN). من غير نت، الموقع هيشتغل عادي، بس الأيقونات مش هتظهر والخط هيتغير لخط النظام.

CDN (Content Delivery Network): سيرفرات عامة بتقدّم ملفات المكتبات المشهورة، فمش لازم تنزّلها جوه المشروع.

11.3 الوضع الغامق اشتغل إزاي؟

خطوتين:

1. في `_Layout.cshtml` : `<html lang="en" data-bs-theme="dark">` . ده بيقول لـ Bootstrap 5.3 يستخدم ألوانه الغامقة.
2. في `site.css` : بنغيّر ألوان Bootstrap الغامقة لألواننا عن طريق **CSS Variables** (الكود تحت).

```
[data-bs-theme="dark"] {
    --bs-body-bg: var(--bg);
    --bs-body-color: var(--text);
    --bs-border-color: var(--border);
    ...
}
```

CSS Variable: قيمة ليها اسم يبدأ بـ `--` ، تقدر تستخدمها في أي مكان بـ `var(--اسم)` . كل ألوان الموقع متعرّفة مرة واحدة في `:root` في أول `site.css` :

```
:root {
    --bg: #0c0d0f;          /* page background (almost black) */
    --panel: #151619;       /* cards and the sidebar */
    --green: #8be36b;       /* main accent color (buttons, active menu) */
    --purple: #7b61ff;      /* second accent color */
    ...
}
```

يعني لو عايز تغيّر اللون الأخضر في الموقع كله، بتغيّره في سطر واحد.

11.4 أجزاء `site.css`

الجزء	الكلاسات	وظيفته
الألوان	`:root` و `[data-bs-theme="dark"]`	كل الألوان، وربطها بـ Bootstrap
تقسيم الصفحة	`.app` و `.sidebar` و `.brand` و `.menu-link` و `.active` و `.main` و `.content`	الـ Sidebar على الشمال والمحتوى على اليمين (`display: flex`)
رأس الصفحة	`.page-header`	العنوان على الشمال والزرار على اليمين
الكروت	`.card` و `.card-header` و `.card-body` و `.form-card`	الصناديق الغامقة. `.form-card` أقصى عرضها 640px
الجداول	`.table` و `.table-sm` و `.small-muted`	عناوين صغيرة رمادي، Hover، ومن غير خط تحت آخر صف
الدوائر	`.avatar` و `.avatar-lg` و `.person`	الدايرة اللي فيها أول حرف من الاسم
الـ Labels	`.pill` و `.pill-green` و `.pill-purple` و `.pill-yellow` و `.pill-red`	زي "3 items" و "Fastest"
الزرار	`.btn-primary` و `.btn-secondary` و `.btn-danger` و `.btn-warning` و `.btn-link`	ألوان الزرار
الفورمز	`.form-control` و `.form-select` و `.text-danger`	خانات غامقة وبوردر أخضر لما تختارها
الرسايل	`.alert-success` و `.alert-danger` و `.alert-warning` و `.alert-info`	رسايل شفافة ملونة
المربعات الكبيرة	`.tile` و `.tile-green` و `.tile-purple` و `.tile-dark` و `.tile-icon`	مربعات الـ Dashboard وصفحة الأداء
الـ Pager	`.pagination`	Previous و Next
صفحة الأداء	`.bar` و `.bar-best` و `.sql-box` و `details summary`	شريط الوقت وصندوق الـ SQL
الموبايل	`@media (max-width: 900px)`	الـ Sidebar بيتحول لشريط فوق

ليه الزرايز متلونة بـ Variables؟ في Bootstrap 5.3، الزرار بياخد ألوانه من متغيرات زي `--bs-btn-bg` و `--bs-btn-hover-bg`. لو غيّرنا `background` على طول، لون الـ Hover كان هيفضل أزرق بتاع Bootstrap، لأن Bootstrap بيحط لون الـ Hover من المتغير. فبنغيّر المتغيرات نفسها:

```
.btn-primary,
.btn-success {
    --bs-btn-bg: var(--green);
    --bs-btn-hover-bg: var(--green-light);
    --bs-btn-color: var(--green-dark);
    ...
}
```

11.5 JavaScript في المشروع

- **مفيش JavaScript خاص بالمشروع.** `site.js` فاضي.
- السطر الوحيد هو `onclick="return confirm('...')"` على زرار "Delete all and generate". لو المستخدم داس Cancel، `confirm` بترجع `false`، والفورم ما بيتبعتش.
- `asp-append-version="true"` على `site.css` و `site.js` بيضيف Hash للرابط (زي `site.css?v=abc123`). لما الملف يتغير، الـ Hash يتغير، فالمتصفح يحمّل النسخة الجديدة بدل القديمة المتخزنة عنده.

11.6 شكل الصفحات على الموبايل

تحت 900px عرض، الـ Sidebar بيتحول لشريط أفقي فوق، والجداول جوه `table-responsive` بتتحرك بالعرض بدل ما تكسر الصفحة.

12. تتبّع أهم السيناريوهات خطوة بخطوة

السيناريو 1: تشغيل الموقع لأول مرة

1. `Main` في `Program.cs` بيشتغل، وبيسجّل MVC و `AppDbContext` .
2. `builder.Build()` بيبني التطبيق.
3. بيعمل Scope ويطلب `AppDbContext` ، ويسأل `db.Customers.Any()` .
4. لو مفيش عملاء: `SeedData.AddTestData(db, 100)` بيضيف 100 عميل وطلباتهم وعناصرهم بـ `SaveChanges()` واحدة.
5. الـ Pipeline بيتبني، و `app.Run()` بيبدأ يستقبل Requests.
6. المتصفح بيفتح `/` ← `HomeController.Index()` ← `Views/Home/Index.cshtml` جوه `_Layout` .

السيناريو 2: عرض قائمة الطلبات والتنقل بين الصفحات

1. المستخدم بيدوس **Orders** في الـ Sidebar ← `GET /Orders` .
2. Routing ← `OrdersController.Index(page = 1)` .
3. `CountAsync()` ← استعلام `SELECT COUNT(*)` ← مثلاً 556 طلب ← 28 صفحة.
4. `Math.Clamp(page, 1, totalPages)` : لو حد كتب `?page=999` ، بيرجع لآخر صفحة.
5. استعلام Projection واحد: أحدث 20 طلب، ومعاهم اسم العميل (`JOIN`) وعدد العناصر (`COUNT`) والإجمالي (`SUM`)، كله محسوب في SQL Server.
6. `ViewBag.Page = 1` و `ViewBag.TotalPages = 28` .
7. `Views/Orders/Index.cshtml` بيرسم الجدول، و `_Pager` بيرسم "Page 1 of 28" وزرار Next.
8. Next ← `GET /Orders?page=2` ← نفس الخطوات بـ `Skip(20)` .

السيناريو 3: عرض تفاصيل طلب

1. أيقونة العين ← `GET /Orders/Details/5856` .
2. `OrdersController.Details(5856)` .
3. استعلام واحد بـ `Include(o => o.Customer).Include(o => o.Items)` و `AsNoTracking()` .
4. لو `order == null` ← `NotFound()` ← صفحة 404.
5. غير كده ← `Views/Orders/Details.cshtml` : كارت العميل، وجدول العناصر، و `orderTotal` بيتحسب في الـ View بـ `Sum` .

السيناريو 4: إضافة طلب جديد وعنصر فيه (الطريق الناجح)

1. **New order** ← `GET /Orders/Create` .
2. `Create(customerId = null)` : طلب جديد بتاريخ النهارده و `CustomerId = 0` ، و `FillCustomerListAsync()` بيجيب `Id` و `Name` لكل العملاء.
3. الفورم بيظهر: Dropdown أول اختيار فيه "-- Choose a customer --"، وخانة التاريخ.

4. المستخدم يختار عميل ويدوس **Save** ← `POST /Orders/Create` ومعاه `CustomerId=12&OrderDate=2025-06-15&__RequestVerificationToken=...`.
5. `[ValidateAntiForgeryToken]` يتأكد من الـ Token، و Model Binding يملا `order` (الحقلين اللي في `[Bind]` بس).
6. `CheckOrderAsync` : التاريخ مش في المستقبل، والعميل 12 موجود ← مفيش أخطاء.
7. `_db.Orders.Add(order)` ثم `SaveChangesAsync()` ← `INSERT INTO Orders` ← `order.Id` بيبقى مثلاً 557.
8. `TempData["Success"] = "The order was created. Now add some items to it."` ← Redirect لـ `/Orders/Details/557`.
9. صفحة التفاصيل بتظهر بالرسالة الخضرا و "This order has no items yet.".
10. **Add item** ← `GET /OrderItems/Create?orderId=557` ← فورم فيه `OrderId=557` مخفي.
11. Save ← `POST /OrderItems/Create` ← يتأكد إن الطلب 557 موجود ← الـ Attributes بتتحقق ← `INSERT INTO OrderItems` ← Redirect لـ `/Orders/Details/557`.

السيناريو 5: إضافة طلب ببيانات غلط

1. المستخدم يسيب "-- Choose a customer --" ويختار تاريخ 01-01-2030 ويدوس Save.
2. **في المتصفح:** jQuery Validation بيلاقي إن `CustomerId` قيمته 0، و `[Range(1, ...)]` مش موافق، فبيعرض "Please choose a customer." على طول، **والفورم ما بيتبعتش**.
3. لو JavaScript مقفول والفورم وصل للسيرفر: الـ `[Range]` بيضيف "Please choose a customer."، و `CheckOrderAsync` بيضيف "The order date cannot be in the future.".
4. `ModelState.IsValid == false` ← `FillCustomerListAsync()` تاني (لأن `ViewBag` مش بيفضل بين الطلبات) ← `return View(order)`.
5. الفورم بيرجع بالقيم اللي المستخدم كتبها، وكل رسالة تحت خانتها. **مفيش أي حاجة اتحفظت.**

السيناريو 6: تعديل طلب

1. **Edit order** ← `GET /Orders/Edit/557` ← `FindAsync(557)` + `FillCustomerListAsync()` ← الفورم بالقيم الحالية.
2. Save ← `POST /Orders/Edit/557`.
3. `FindAsync(557)` تاني (لو اتمسح في النص ← 404).
4. `order.Id = id`، و `CheckOrderAsync(order)`.
5. نسخ `OrderDate` و `CustomerId` على `savedOrder` ← `SaveChangesAsync()` ← `UPDATE Orders SET ... WHERE Id = 557` (الأعمدة اللي اتغيرت بس).
6. Redirect لصفحة التفاصيل و "The order was updated.".

السيناريو 7: تعديل عنصر

1. أيقونة القلم جنب العنصر ← `GET /OrderItems/Edit/18650`.
2. Save ← `POST /OrderItems/Edit/18650` ومعاه `ProductName` و `Quantity` و `UnitPrice` بس (`[Bind]`).
3. `savedItem = FindAsync(18650)`، و `item.OrderId = savedItem.OrderId` (من قاعدة البيانات، مش من الفورم).

4. الـ Attributes بتتحقق ← نسخ القيم ← `SaveChangesAsync()` ← Redirect لـ `/Orders/Details/{OrderId}` ، والإجمالي بيتحدث.

السيناريو 8: حذف عنصر

1. أيقونة الزبالة ← `GET /OrderItems/Delete/18650` ← صفحة فيها المنتج والكمية والسعر.
2. **Yes, delete** ← `POST /OrderItems/Delete/18650` ← `DeleteConfirmed` .
3. `FindAsync` ← بيحفظ `orderId` في متغير قبل الحذف ← `Remove` ← `SaveChangesAsync()` ← `DELETE FROM OrderItems WHERE Id = 18650` .
4. Redirect للطلب و "The item was deleted.".

السيناريو 9: حذف طلب

1. **Delete order** ← `GET /Orders/Delete/557` ← الصفحة بتحذّر: "Deleting this order also deletes its 3 item(s).".
2. **Yes, delete** ← `POST /Orders/Delete/557` .
3. `_db.Orders.Remove(order)` ← `SaveChangesAsync()` ← `DELETE FROM Orders WHERE Id = 557` .
4. SQL Server بيمسح عناصر الطلب تلقائي بسبب `ON DELETE CASCADE` .
5. Redirect لـ `/Orders` و "Order #557 was deleted.".

السيناريو 10: محاولة حذف عميل عنده طلبات

1. `GET /Customers/Delete/12` ← `ViewBag.OrderCount = 4` ← الصفحة بتعرض تحذير، و**مفيش** زرار "Yes, delete".
2. لو حد بعت POST بنفسه (مثلاً من أداة)، `DeleteConfirmed` بيعمل `AnyAsync(o => o.CustomerId == 12)` ← `true` ← `TempData["Error"]` ← Redirect لصفحة العميل.
3. ولو حتى الكود ده اتشال، قاعدة البيانات هترفض بسبب `ON DELETE RESTRICT` ، وهيتمسك `DbUpdateException` .

السيناريو 11: فتح طلب مش موجود

1. `GET /Orders/Details/999999` .
2. `FirstOrDefaultAsync` بيرجع `null` ← `return NotFound()` ← الرد 404 من غير محتوى.
3. Middleware `UseStatusCodePagesWithReExecute` بيمسك الـ 404 ← يشغّل `/Home/HttpError?code=404` جوه السيرفر.
4. `HomeController.HttpError(404)` ← `Views/Home/HttpError.cshtml` ← "Not found" وزراير للطلبات والعملاء.
5. الرابط في المتصفح بيفضل `/Orders/Details/999999` والكود 404.

السيناريو 12: تشغيل مقارنة الأداء

1. `GET /Performance` ← `CountRowsAsync` + `LoadHistory` ← الصفحة بالمربعات والفورمز والنتايج القديمة.
2. المستخدم يكتب 200 طلب و 5 تشغيلات ويدوس **Run test** ← `POST /Performance/Run` .
3. `[Range]` على `PerformanceViewModel` بيتحقق من الأرقام.
4. لكل طريقة من الـ 3، `MeasureAsync` :
    1. تشغيلة رقم 0 (تسخين، مش بتتقاس) + 5 تشغيلات مقيسة.

2. قبل كل تشغيلة: `_db.ChangeTracker.Clear()` ، وبعدين `Stopwatch` حوالين التحميل.
3. الطريقة نفسها بتحفظ نص الـ SQL (من `ToQueryString()`) وعدد الاستعلامات اللي عدّته.
4. بيحفظ الوقت وعدد الطلبات والإجمالي.

5. بيقارن كل طريقة بالأولى (`SameDataAsFirst`).
6. `SaveResults` ← 3 سطور جديدة في `perf-results.csv` .
7. `View("Index", model)` ← جدول النتايج، والأسرع عليه "Fastest"، وشرائط بتوضح الوقت.

السيناريو 13: إنشاء بيانات تجريبية

1. المستخدم يكتب مثلاً 1000 ويدوس **Delete all and generate** ← رسالة `confirm` في المتصفح ← OK.
2. `POST /Performance/Generate` ← `[Range(10, 5000)]` .
3. `RemoveRange` على العناصر ثم الطلبات ثم العملاء، وبعدين `SaveChangesAsync()` ← أوامر `DELETE` لكل الصفوف.
4. `SeedData.AddTestData(_db, 1000)` ← `SaveChanges()` .
5. Redirect لـ `/Performance` و "New test data was created with 1000 customers.".
6. لو حصل `DbUpdateException` ، بتظهر رسالة "The test data could not be created. Please try again.". ومفيش Transaction، فلو الخطأ حصل بعد المسح، البيانات القديمة بتكون اتمسحت (وده مقبول لأنها أداة اختبار).

13. شرح أهم الأكواد سطر بسطر

13.1 `Program.cs` كله

السطور	الكود	الشرح
12	`var builder = WebApplication.CreateBuilder(args);`	بيجهز التطبيق: بيقرا `appsettings.json` و`launchSettings`، وبيجهز الـ Logging و
14	`builder.Services.AddControllersWithViews();`	بيضيف MVC (Controllers + Views)
16	`string connectionString = builder.Configuration.GetConnectionString("DefaultConnection")!;`	بيقرا عنوان قاعدة البيانات
18	`builder.Services.AddDbContext<AppDbContext>(options => options.UseSqlServer(connectionString));`	بيسجّل الـ DbContext في الـ DI ويربطه بـ SL Server
20	`var app = builder.Build();`	بيبني التطبيق. بعد السطر ده مش ممكن تض Services جديدة
22 لـ 30	`using (var scope = ...) { ... SeedData.AddTestData(db, 100); }`	بيضيف 100 عميل لو قاعدة البيانات فاضية
32 لـ 37	`if (!app.Environment.IsDevelopment()) { UseExceptionHandler; UseHsts; }`	في Production بس: صفحة خطأ عامة، وإج المتصفح على HTTPS
39	`app.UseStatusCodePagesWithReExecute("/Home/HttpError", "?code={0}");`	أي 404 (أو كود خطأ تاني من غير محتوى) يع صفحة `HttpError`
41	`app.UseHttpsRedirection();`	يحوّل HTTP لـ HTTPS (لو فيه بورت TTPS معروف)
43	`app.UseRouting();`	يحدد الـ Endpoint (Controller/Action) ا للرابط
45	`app.UseAuthorization();`	من القالب. مش بيعمل حاجة فعلية لأن مفي `[Authorize]`
47	`app.MapStaticAssets();`	يقدّم ملفات `wwwroot` بطريقة محسّنة (مو .NET 9)
49 لـ 52	`app.MapControllerRoute(...)`	الـ Route الافتراضي `ller=Home}/{action=Index}/{id?}`
54	`app.Run();`	يبدأ السيرفر ويستنى Requests

ترتيب الـ Middleware مهم. مثلاً `UseRouting` لازم يبقى قبل `UseAuthorization`، لأن الـ Authorization محتاج يعرف أنهي Endpoint اتختار.

13.2 استعلام قائمة الطلبات (`OrdersController.Index` ، سطور 35 لـ 48)

```
List<OrderListItem> orders = await _db.Orders
    .OrderByDescending(o => o.OrderDate)
    .ThenByDescending(o => o.Id)
    .Skip((page - 1) * PageSize)
    .Take(PageSize)
    .Select(o => new OrderListItem
    {
        Id = o.Id,
        OrderDate = o.OrderDate,
        CustomerName = o.Customer!.Name,
        ItemCount = o.Items.Count,
        Total = o.Items.Sum(i => i.Quantity * i.UnitPrice)
    })
    .ToListAsync();
```

السطر	الشرح
`_db.Orders`	ابدأ من جدول الطلبات
`.OrderByDescending(o => o.OrderDate)`	الأحدث الأول (الـ Index على `OrderDate` بيخليها سريعة)
`.ThenByDescending(o => o.Id)`	لو طلبين في نفس اليوم، رتّبهم بالرقم. ده بيخلي الترتيب **ثابت**، فمفيش طلب يظهر في صفحتين أو يختفي وانت بتقلّب
`.Skip((page - 1) * PageSize)`	اتخطى الصفحات اللي فاتت. صفحة 3 = اتخطى 40
`.Take(PageSize)`	خد 20 بس
`.Select(o => new OrderListItem { ... })`	**Projection**: ما تجيبش الطلب كله، كوّن صف صغير فيه 5 قيم بس
`CustomerName = o.Customer!.Name`	EF بيحوّلها لـ `JOIN` مع `Customers`. علامة `!` للكومبايلر بس، ملهاش تأثير على SQL
`ItemCount = o.Items.Count`	بيتحول لـ `COUNT(*)` في Subquery جوه SQL Server
`Total = o.Items.Sum(i => i.Quantity * i.UnitPrice)`	بيتحول لـ `SUM(Quantity * UnitPrice)` جوه SQL Server. لو مفيش عناصر، EF بيرجّع 0 مش null
`.ToListAsync()`	هنا بس الاستعلام بيتبعت، والنتيجة `List<OrderListItem>`

النتيجة: **استعلام واحد** بيرجّع 20 صف صغير، بدل ما نجيب 20 طلب + عملائهم + كل عناصرهم.

13.3 تعديل عنصر (`OrderItemsController.Edit` POST، سطور 95 لـ 132)

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, [Bind("ProductName,Quantity,UnitPrice")] OrderItem
item)
{
    OrderItem? savedItem = await _db.OrderItems.FindAsync(id);
    if (savedItem == null)
    {
        return NotFound();
    }

    item.Id = id;
    item.OrderId = savedItem.OrderId;

    if (!ModelState.IsValid)
    {
        return View(item);
    }

    savedItem.ProductName = item.ProductName;
    savedItem.Quantity = item.Quantity;
    savedItem.UnitPrice = item.UnitPrice;

    try
    {
        await _db.SaveChangesAsync();
        TempData["Success"] = "The item was updated.";
        return RedirectToAction("Details", "Orders", new { id = savedItem.OrderId });
    }
    catch (DbUpdateException ex)
    {
        _logger.LogError(ex, "Could not update item {ItemId}.", id);
        ModelState.AddModelError("", "The changes could not be saved. ...");
        return View(item);
    }
}
```

السطر	الشرح
`int id`	جاي من الرابط `/OrderItems/Edit/18650`
`[Bind("ProductName,Quantity,UnitPrice")] OrderItem item`	جاي من الفورم، بس الـ 3 حقول دول. `Id` و `OrderId` **مش** بيتاخدوا من الفورم
`FindAsync(id)`	هات العنصر المحفوظ. الـ DbContext بيبدأ يتابعه (Tracking)
`if (savedItem == null) return NotFound();`	لو اتمسح في الوقت ده، 404
`item.Id = id;`	عشان الفورم لو رجع بأخطاء، رابطه يبقى صح
`item.OrderId = savedItem.OrderId;`	الطلب الحقيقي من قاعدة البيانات، عشان زرار Cancel يرجع للطلب الصح، ومحدش يقدر ينقل العنصر
`if (!ModelState.IsValid) return View(item);`	اتحققت (`[Range(1, 1000)]` زي `OrderItem` اللي في الـ Attributes) تلقائي قبل الـ Action. لو فيه أخطاء، ارجع للفورم ومتحفظش حاجة

السطر	الشرح
`savedItem.ProductName = ...` (3 سطور)	انسخ القيم الجديدة على الكائن المتتبَّع. EF بيعلّم الأعمدة اللي اتغيرت فعلاً
`await _db.SaveChangesAsync();`	`UPDATE OrderItems SET ... WHERE Id = @id`
`RedirectToAction("Details", "Orders", ...)`	ارجع لصفحة الطلب (PRG)
`catch (DbUpdateException ex)`	أي مشكلة في قاعدة البيانات: سجّلها وقول للمستخدم رسالة بسيطة

13.4 عدّ الاستعلامات في طريقة N+1 (`PerformanceController.LoadWithNPlusOneAsync`، سطور 152 لـ 181)

```
IQueryable<Order> query = _db.Orders
    .OrderByDescending(o => o.OrderDate)
    .ThenByDescending(o => o.Id)
    .Take(count);

if (result.Sql == "")
{
    result.Sql = query.ToQueryString();
}

List<Order> orders = await query.ToListAsync();
result.QueryCount = 1;

foreach (Order order in orders)
{
    if (!_db.Entry(order).Reference(o => o.Customer).IsLoaded)
    {
        await _db.Entry(order).Reference(o => o.Customer).LoadAsync();
        result.QueryCount++;
    }

    await _db.Entry(order).Collection(o => o.Items).LoadAsync();
    result.QueryCount++;
}

return ToListItems(orders);
```

الجزء	الشرح
`IQueryable<Order> query = ...Take(count);`	بنبني الاستعلام الأول بس (الطلبات من غير العملاء والعناصر). لسه ما اتنفذش
`if (result.Sql == "") result.Sql = query.ToQueryString();`	`ToQueryString()` بترجّع نص الـ SQL من غير ما تنفذه. بنحفظه مرة واحدة بس (في تشغيلة التسخين)، عشان الصفحة تعرضه
`await query.ToListAsync()`	هنا الاستعلام الأول بيتنفذ
`result.QueryCount = 1;`	بنعدّ الرحلات لقاعدة البيانات بإيدينا: 1 لحد دلوقتي
`_db.Entry(order).Reference(o => o.Customer).IsLoaded`	هل عميل الطلب ده اتحمّل خلاص؟ لو طلب تاني لنفس العميل حمّله قبل كده، EF بيعرف وبيرجّع `true`
`QueryCount++` + `Reference(...).LoadAsync()`	**Explicit Loading**: حمّل العميل (استعلام جديد) وزوّد العدّاد

الجزء	الشرح
`+ Collection(o => o.Items).LoadAsync()` `QueryCount++`	حمّل عناصر الطلب (استعلام جديد لكل طلب، دايماً)
`return ToListItems(orders);`	حوّل الطلبات لصفوف القائمة، زي باقي الطرق بالظبط

ليه بنعدّ بإيدينا؟ عشان الكود يفضل بسيط ومفهوم: كل مرة بنكلم فيها قاعدة البيانات، بنزوّد الرقم. والطريقتين التانيين (`Include` و `Select`) بيعملوا استعلام واحد بس، فالعدد عندهم `1` على طول. واتأكدت من صحة العدّ: الصفحة طلّعت 281 على 200 طلب، وده نفس الرقم اللي استعلام SQL مباشر بيدّيه (1 + 200 + 80 عميل مختلف).

13.5 القياس (`PerformanceController.MeasureAsync` ، سطور 105 لـ 148)

```
for (int run = 0; run <= runs; run++)
{
    _db.ChangeTracker.Clear();

    var stopwatch = Stopwatch.StartNew();

    List<OrderListItem> rows;
    if (way == NPlusOne)
    {
        rows = await LoadWithNPlusOneAsync(ordersToLoad, result);
    }
    else if (way == IncludeWay)
    {
        rows = await LoadWithIncludeAsync(ordersToLoad, result);
    }
    else
    {
        rows = await LoadWithSelectAsync(ordersToLoad, result);
    }

    stopwatch.Stop();

    if (run == 0)
    {
        continue;
    }

    times.Add(stopwatch.Elapsed.TotalMilliseconds);
    result.OrdersLoaded = rows.Count;
    result.GrandTotal = rows.Sum(r => r.Total);
}
result.AverageMs = times.Average();
result.FastestMs = times.Min();
result.SlowestMs = times.Max();
```

السطر	الشرح
`run <= runs`	لو `runs = 5` ، الحلقة بتلف 6 مرات (من 0 لـ 5)
`_db.ChangeTracker.Clear()`	بيخلّي الـ DbContext ينسى كل الكائنات اللي اتحمّلت في التشغيلة اللي فاتت، عشان كل التشغيلات تبدأ من نفس الظروف

السطر	الشرح
`Stopwatch.StartNew()` و `Stop()`	ساعة دقيقة بتقيس وقت التحميل بس
`if / else if / else`	بيختار الطريقة حسب اسمها
`if (run == 0) continue;`	التشغيلة الأولى **تسخين**: أول مرة EF بيجهز الاستعلام، فبتبقى أبطأ ومش عادلة. فبنرميها
`times.Average()` / `Min()` / `Max()`	LINQ على `List<double>` : المتوسط والأسرع والأبطأ

13.6 الدايرة والـ Label في قائمة الطلبات (`Views/Orders/Index.cshtml`)

```
<span class="avatar">@(string.IsNullOrEmpty(order.CustomerName) ? "?" :
order.CustomerName.Substring(0, 1).ToUpper())</span>
<span>@order.CustomerName</span>
...
<td><span class="pill pill-purple">@order.ItemCount @(order.ItemCount == 1 ? "item" : "items")
</span></td>
<td class="text-end fw-semibold">@order.Total.ToString("C")</td>
```

الجزء	الشرح
`@( ... )`	تعبير C# كامل جوه HTML. لازم الأقواس لما التعبير فيه مسافات أو `?`
`string.IsNullOrEmpty(...) ? "?" : ...`	لو الاسم فاضي، اكتب `?` ، عشان `Substring` ما ترميش Exception
`.Substring(0, 1).ToUpper()`	أول حرف كبير: "C" ← "Customer 944"
`order.ItemCount == 1 ? "item" : "items"`	"item 1" و "items 3"
`ToString("C")`	شكل فلوس حسب إعدادات Windows (عندك en-US): `$2,643.91`
`class="avatar"` و `class="pill pill-purple"`	كلاسات من `site.css`

14. الأخطاء اللي اتصلحت، والقرارات التقنية، والأجزاء غير المكتملة

14.1 الأخطاء والمشاكل اللي اتصلحت

#	المشكلة	السبب	الحل
1	تحذير أمني NU1903 وقت تثبيت SQLite	نسخة `Microsoft.EntityFrameworkCore.Sqlite` 10.0.0 كانت بتجيب `SQLitePCLRaw.lib.e_sqlite3` 2.1.11 وفيها ثغرة معروفة	الترقية لـ 10.0.13. وبعدين SQLite Server
2	SQLite مش بيقدر يعمل `SUM` على `decimal`	SQLite مفيهوش نوع decimal	مؤقتاً: `Conversion<double>()` `HasPrecision(18, 2)`
3	إنشاء البيانات التجريبية كان بطيء	EF كان بيطبع كل `INSERT` في الـ Console	`orkCore.Database.Command":` `appsettings` في `"Warning"`
4	شرح N+1 كان غلط	الشرح كان بيقول "1 + 2 لكل طلب" (401)، بس القياس طلع 281	تعديل الشرح لـ "لحد 2 لكل طلب"، اتحمّل قبل كده
5	ثغرتين (NU1902 و NU1903) في `System.Security.Cryptography.Pkcs` 9.0.11	جاية مع `Microsoft.Data.SqlClient` 6.1.7 (تبع SQL Server provider)	إضافة `ty.Cryptography.Pkcs` صراحة في `.csproj`
6	`EnsureCreated()` ما ينفعش مع الـ Migrations	`EnsureCreated` بيعمل الجداول من غير ما يسجّل في `__EFMigrationsHistory`، فأي Migration بعد كده تفشل	شيلناه، والجداول بقت بتتعمل بـ `e`
7	**InvalidOperationException: The "view 'Index' was not found"** في صفحة الطلبات	فولدر `Views/Orders` اتنقل بالغلط جوه `Views/OrderItems` (غالباً بالسحب في Solution Explorer)، و MVC بيدور في `Views/Orders` بس	رجّعنا الفولدر لمكانه
8	رسايل تحقق إنجليزي تلقائية فيها أسماء Properties (زي "The UnitPrice field is required.")	بعد شيل الـ Attributes، MVC لسه بيضيف تحقق في المتصفح لخانات الأرقام والتواريخ	شيل سكريبتات التحقق من الـ 6 فور ورسايل Model Binding مخصصة
9	رسالتين خطأ على خانة الكمية لما تبقى فاضية	Model Binding بيضيف خطأ، والـ Controller كان بيضيف خطأ النطاق كمان	شرط `State(...) != Invalid`
10	Labels بتعرض أسماء Properties (UnitPrice) وخانة التاريخ بقت تاريخ ووقت	`[Display]` و `[DataType(DataType.Date)]` اتشالوا	كتابة نص الـ Label في الـ View، و `'`
11	ألوان اللينكات والزراير غلط بعد التصميم الغامق	ملف `_Layout.cshtml.css` اللي جاي مع القالب كان بيفرض ألوان زرقا	مسح الملف والـ Link بتاعه من الـ `t`
12	"1 items"	جمع ثابت	`t == 1 ? "item" : "items"`
13	جدول نتايج الأداء عريض، والأسماء بتتكسر على 3 سطور	10 أعمدة بمسافات كبيرة	كلاس `table-sm` و `xt-nowrap`

#	المشكلة	السبب	الحل
14	أجزاء في الكود متقدمة وصعبة على حد لسه مخلص MVC	`QueryCounter` (Interceptor)، و Transaction يدوي، و `ExecuteDeleteAsync` ، و 5 طرق مقارنة، و Regex، وإعدادات MVC خاصة في `Program.cs` ، و `UseRequestLocalization` ، و `GetFieldValidationState`	تبسيط الكود: رجوع Annotations والعدّ بإيدينا، و `racker.Clear()` و `DbSet` بـ `{ get; set; }` . و
15	نص خطأ (`CS1061: 'PerformanceResult' does not contain a definition for 'SqlStatements'`) ومعاه اقتراح كود اتلزقوا جوه `Views/Performance/Index.cshtml`	بعد ما `SqlStatements` اتشالت من الـ Model، الـ View كانت لسه بتستخدمها. Visual Studio طلّع الخطأ، والنص ده اتلزق في الملف بالغلط (غالباً من اقتراح تصحيح في Visual Studio)	إعادة كتابة الـ View على الـ Model

ملحوظة: المشاكل 8 و 9 و 10 حصلت في مرحلة كانت الـ Attributes فيها متشالة من الـ Models. بعد التبسيط (رقم 14)، الـ Attributes رجعت، فالحلول المؤقتة دي اتشالت هي كمان.

14.2 القرارات التقنية المهمة

القرار	السبب	البديل اللي ما اخترناهوش
ASP.NET Core MVC بصفحات بتترسم على السيرفر	ده المطلوب في المشروع، وأبسط من فصل Frontend و API	Razor Pages، أو API + React/Angular
SQL Server Express بـ Windows Authentication	قاعدة البيانات اللي موجودة على جهازك، ومفيش باسورد في الكود. المشروع بدأ بـ SQLite وبعدين اتنقل	SQLite (ملف واحد، أسهل في النقل)، أو LocalDB
مفيش Service ولا Repository Layer	المشروع صغير، و `DbContext` نفسه بيعمل شغل Repository و Unit of Work. ده بيخلي الكود أقصر وأسهل للمبتدئ	Services عشان الـ Controllers تبقى أخف وأسهل في الـ Unit Testing
Fluent API لقواعد قاعدة البيانات، و Data Annotations للتحقق	ده اللي بيتعلّم في الكورسات: كل أداة في مكانها. ونفس الـ Attribute بيتحقق في المتصفح وعلى السيرفر	التحقق اليدوي في الـ Controllers (كان موجود قبل كده، واتشال لأنه أطول وأصعب على المبتدئ)
Paging بـ 20 صف	القوائم تفضل سريعة مهما البيانات كترت	تحميل كل حاجة مرة واحدة (بطيء مع آلاف الصفوف)
Projection للقوائم، و `Include` + `AsNoTracking` للتفاصيل	دي الطرق اللي قياس صفحة الأداء أثبت إنها الأسرع لكل حالة	Include في كل حاجة، أو Lazy Loading
`Restrict` للعملاء و `Cascade` للعناصر	حماية سجل الطلبات، والعنصر ملوش معنى من غير طلبه	Cascade في الاتنين (خطر)، أو Soft Delete (علامة "ممسوح" بدل المسح)
Unique Index على الإيميل + فحص في الكود	الكود بيدّي رسالة واضحة، و الـ Index بيحمي لو طلبين وصلوا في نفس اللحظة	الفحص في الكود بس

القرار	السبب	البديل اللي ما اخترناهوش
Load-Copy-Save في التعديل	أمان ضد Overposting، وتحديث الأعمدة اللي اتغيرت بس	`_db.Update(model)`
PRG + `TempData`	مفيش إرسال مكرر للفورم، والرسالة بتظهر بعد الـ Redirect	عرض View بعد POST
`ViewBag` لـ Pager والـ Dropdown	أبسط من عمل View Model لكل صفحة	View Models مخصصة (أقوى لأن فيها Types)
صفحة 404 بـ `UseStatusCodePagesWithReExecute`	كل `NotFound()` في المشروع بيعرض نفس الصفحة الحلوة من غير تكرار كود	View خاصة لكل Controller
الاعتماد على إعدادات Windows للأرقام والفلوس	أبسط، وجهازك أصلاً en-US (نقطة عشرية و `$`)	تثبيت en-US في الكود بـ `UseRequestLocalization` (كان موجود واتشال للتبسيط)
تعليق قصير فوق كل Class و Method و View، بدل تعليق فوق كل سطر	الكود بقى أنضف وأسهل في القراية، والتعليق بيقول الحاجة بتعمل إيه وبتتنده منين	تعليق فوق كل سطر (كان موجود قبل كده)
3 طرق بس في صفحة الأداء	دول الأساسيين اللي بيتشرحوا في أي كورس EF Core، وكفاية عشان نوضح الفرق	5 طرق (كانت فيها Split Query و Include من غير `AsNoTracking`)
`Random(12345)` + تسخين + `ChangeTracker.Clear()` قبل كل تشغيلة + CSV	عشان القياس يبقى ممكن يتكرر تحت نفس الظروف، ويتسجل	قياس مرة واحدة من غير تسجيل
مفيش عمود Total	الإجمالي بيتحسب من العناصر، فمستحيل يختلف عنها	عمود Total لازم يتحدث مع كل تغيير
اسم المنتج نص، من غير جدول Products	المطلوب 3 Models بس	جدول Products وعلاقة كمان

14.3 الأجزاء غير المكتملة والقيود (بصراحة)

وظايف ناقصة:

- مفيش Authentication ولا Authorization، فأي حد يقدر يعدّل ويمسح كل حاجة.
- مفيش بحث ولا فلترة ولا ترتيب حسب اختيار المستخدم.
- مفيش Unit Tests ولا Integration Tests.
- مفيش حماية من التعديل المتزامن (Concurrency Token): لو اتنين عدّلوا نفس العميل في نفس الوقت، آخر واحد يحفظ هو اللي بيكسب.
- صفحة Privacy فيها نص مبدئي من القالب.
- صفحة تفاصيل العميل بتعرض أرقام وتواريخ الطلبات بس، من غير الإجماليات.
- الـ Dropdown في فورم الطلب بيحمّل **كل** العملاء (ممكن يبقوا آلاف) ومفيهوش بحث.

تفاصيل تقنية:

- زرار Generate بيمسح **كل** البيانات، ومحمي بـ `confirm` في المتصفح بس. ده أداة اختبار ولازم ما يبقاش موجود في نسخة حقيقية.
- `Generate` من غير Transaction: لو إنشاء البيانات الجديدة فشل بعد المسح، البيانات القديمة بتكون اتمسحت. وبيمسح بـ `RemoveRange`، يعني EF بيحمّل الصفوف الأول وبعدين يمسحها، فمع آلاف الصفوف بياخد شوية ثواني.
- `Run` في صفحة الأداء بيرجّع View بعد POST، فالـ Refresh بيعيد الاختبار.
- البيانات التجريبية عند التشغيل محتاجة `Update-Database` يكون اتعمل الأول.
- `TrustServerCertificate=True` مناسب للتطوير بس.
- قائمة العملاء مترتبة بالاسم كنص، فـ "Customer 10" بييجي قبل "Customer 2".
- `OrderDate` متخزن `datetime2` (تاريخ ووقت) مع إن الواجهة بتستخدم التاريخ بس.
- `[EmailAddress]` و `[Phone]` بيعملوا تحقق بسيط، ومش بيغطوا كل الحالات.
- عدد الاستعلامات في طريقة N+1 بيتعد بإيدينا في الكود. والقياس على جهاز واحد، وبيتأثر بأي حمل تاني عليه.
- شكل الأرقام والفلوس بيعتمد على إعدادات Windows. عندك en-US، فالكسور بنقطة والعملة `$`. على جهاز إعداداته مختلفة، الشكل ممكن يتغير.

ملفات ممكن تتنضف:

- `sales.db`: من أيام SQLite، ومش مستخدم.
- `perf-results.csv` في فولدر المشروع: ممكن يتنقل لفولدر بيانات.
- Bootstrap JS بيتحمّل ومش مستخدم، و `site.js` فاضي.
- الأيقونات والخط محتاجين نت.

14.4 حاجات مش ممكن أتأكد منها من الكود

- **البيانات اللي في قاعدة البيانات عندك دلوقتي:** عددها بيتغير مع كل استخدام لزرار Generate أو الإضافة والحذف.
- **نسخة SQL Server بالظبط:** الكود بيقول بس إن الـ Instance اسمه `SQLEXPRESS`.
- **الـ Collation بتاع قاعدة البيانات:** ده بيحدد هل مقارنة الإيميل حساسة لحالة الحروف (`A@x.com` و `a@x.com`). الافتراضي في تثبيتات SQL Server الإنجليزي غالباً مش حساس لحالة الحروف، بس مش ممكن أتأكد من إعداد جهازك من الكود.
- **تاريخ التعديلات:** فولدر المشروع مش Git Repository، فمفيش تاريخ Commits أرجع له. ترتيب الأخطاء والقرارات في القسم 14 من سجل الشغل على المشروع.
- **أي نشر على سيرفر حقيقي:** مفيش أي إعدادات نشر (Deployment) في المشروع.

15. أسئلة Interview متوقعة مع الإجابات

أسئلة عامة عن المشروع

س1: اشرح المشروع في دقيقة.

ج: ده تطبيق ASP.NET Core MVC على NET 10. لإدارة العملاء وطلبات المبيعات وعناصر الطلبات، وقاعدة البيانات SQL Server من خلال EF Core. فيه CRUD كامل للتلات Entities، وقوائم بصفحات، وصفحات تفاصيل بتعرض البيانات المرتبطة. ركّزت على 3 حاجات: الأداء (قوائم بـ Projection و Paging و Indexes)، والتحقق من البيانات بـ Data Annotations في المتصفح وعلى السيرفر، والتعامل مع الأخطاء من غير ما أكشف تفاصيل تقنية. وعملت صفحة بتقارن 3 طرق لتحميل البيانات المرتبطة (N+1 و Include و Select)، بتقيس الوقت وعدد استعلامات SQL وبتسجّل النتايج. القياس بيّن إن N+1 أبطأ بحوالي 70 مرة من Select في قائمة الطلبات، فاستخدمت Select للقوائم و Include للتفاصيل.

س2: يعني إيه MVC، وإيه فايدته؟

ج: Model للبيانات (زي `Customer`)، و View للعرض (`.cshtml`)، و Controller بيستقبل الطلب وينسّق بينهم (زي `CustomersController`). الفايدة إن كل جزء له مسؤولية واحدة، فالكود أسهل في الفهم والتعديل. مثلاً لو غيّرت تصميم صفحة العملاء، مش بلمس الـ Controller.

س3: إيه اللي بيحصل من ساعة ما المستخدم يكتب `/Orders` لحد ما الصفحة تظهر؟

ج: الـ Request بيعدّي على الـ Middleware بالترتيب اللي في `Program.cs` . `UseRouting` بيطابق الرابط مع الـ pattern `{controller=Home}/{action=Index}/{id?}` ، فيوصل لـ `OrdersController.Index` . ASP.NET Core بيعمل الـ Controller ويدّيله `AppDbContext` بالـ DI. الـ Action بتعمل `CountAsync` ، وبعدين استعلام Projection لـ 20 طلب. النتيجة `List<OrderListItem>` بتروح للـ View، والـ View بترسم جوه `_Layout` ، والـ HTML يرجع للمتصفح.

س4: يعني إيه Dependency Injection، واستخدمته فين؟

ج: إن الكلاس ما يعملش الحاجات اللي محتاجها بنفسه، لكن ياخدها من برة. سجّلت `AppDbContext` في `Program.cs` بـ `AddDbContext` ، وكل Controller بيطلبه في الـ Constructor، ومعاه `ILogger<T>` . و `PerformanceController` كمان بياخد `IWebHostEnvironment` عشان يعرف مكان فولدر المشروع ويحفظ فيه ملف CSV. الفايدة إن الـ Controller مش مسؤول عن إنشاء الاتصال ولا قفله، وممكن نبدّل التنفيذ في الاختبارات.

س5: الـ DbContext بيعيش قد إيه؟ وليه مش Singleton؟

ج: Scoped، يعني نسخة جديدة لكل Request، وده الافتراضي في `AddDbContext` . مش Singleton لأن `DbContext` مش Thread-safe، وفيه Change Tracker بيحتفظ بالكائنات. لو اتشارك بين المستخدمين، البيانات هتتلخبط والذاكرة هتكبر على طول.

أسئلة EF Core وقاعدة البيانات

س6: يعني إيه ORM، وليه EF Core؟

ج: ORM بيربط الكلاسات بالجداول، فبكتب LINQ بدل SQL. EF Core كمان بيدّيني Migrations لإدارة شكل قاعدة البيانات، وChange Tracking للحفظ، وبيعمل Parameters تلقائي في الاستعلامات، فبيحمي من SQL Injection.

س7: استخدمت Fluent API ولا Data Annotations؟ وليه؟

ج: الاتنين، وكل واحد في مكانه. الـ Fluent API في `OnModelCreating` جوه `AppDbContext` لقواعد قاعدة البيانات: أسماء الجداول، والـ Keys، والعلاقات، و `OnDelete(DeleteBehavior.Restrict)` ، و Unique Index على الإيميل. والـ Data Annotations فوق الـ Properties للتحقق من الفورم: `[Required]` و `[StringLength]` و `[Range]` و `[EmailAddress]` ، ومعاها `[Display]` لأسماء الـ Labels. ميزة الـ Annotations إن نفس القاعدة بتتحقق في المتصفح وعلى السيرفر. وبعد ما ضفتها، اتأكدت بـ `dotnet ef migrations has-pending-model-changes` إن شكل قاعدة البيانات ما اتغيرش.

س8: إيه هي الـ Migrations؟ ولو عايز تضيف عمود؟

ج: ملفات بتسجّل تغييرات شكل قاعدة البيانات في الكود. عندي Migration واحدة اسمها `InitialCreate` عملت الـ 3 جداول والـ Indexes. عشان أضيف عمود: أضيف الـ Property في الـ Model، وقواعده في `OnModelCreating` لو محتاج، وبعدين `Add-Migration AddCustomerAddress` ، وبعدين `Update-Database` . و EF بيسجّل كل Migration اتطبقت في جدول `__EFMigrationsHistory` .

س9: مشكلة N+1 إيه؟ وإزاي اكتشفتها؟

ج: إنك تجيب N سجل في استعلام، وبعدين تعمل استعلام لكل سجل عشان تجيب البيانات المرتبطة. في صفحة الأداء، تحميل 200 طلب بطريقة N+1 عمل 281 استعلام وخد حوالي 431 ms، بينما الـ Select عمل استعلام واحد في حوالي 6 ms. العدد بيتحسب في الكود نفسه: كل مرة بنكلم فيها قاعدة البيانات بنزوّد عدّاد، واتأكدت من الرقم باستعلام SQL مباشر. الحل إني أجيب كل اللي محتاجه في استعلام واحد، بـ `Include` أو `Select` .

س10: الفرق بين `Include` و `Select` ؟

ج: `Include` بيجيب الـ Entity كاملة ومعاها الـ Entities المرتبطة كاملة. مناسب لصفحة تفاصيل الطلب، لأني محتاج كل بيانات العميل وكل عنصر. `Select` (Projection) بيجيب الأعمدة اللي أنا محدّدها بس، وممكن قاعدة البيانات تحسب فيه حاجات زي `COUNT` و `SUM` . مناسب لقائمة الطلبات، لأني محتاج 5 قيم بس لكل طلب.

س11: `AsNoTracking` بيعمل إيه؟

ج: بيقول لـ EF ما يتابعش الكائنات اللي جاية، فمش بيحفظ نسخة منها في الـ Change Tracker. أسرع وأخف على الذاكرة. استخدمته في كل صفحات العرض. في الاختبار، Include مع AsNoTracking خد حوالي 28 ms، ومن غيره حوالي 47 ms. ما استخدمتهوش في التعديل، لأني محتاج الـ Tracking عشان `SaveChangesAsync` يعرف إيه اللي اتغير.

س12: امتى تستخدم Split Query؟

ج: لما يكون فيه `Include` لقائمة كبيرة، فالـ JOIN بيكرر بيانات الطلب والعميل مع كل عنصر. `AsSplitQuery()` بيقسمها لاستعلامين أصغر. **مش مستخدمة في المشروع دلوقتي**، لأن كل طلب فيه من 1 لـ 5 عناصر بس، فالفرق صغير. كانت موجودة في نسخة قبل كده من صفحة الأداء، وكانت قريبة من Include في السرعة.

س13: ليه ما استخدمتش Lazy Loading؟

ج: لأنه بيحمّل البيانات المرتبطة وقت ما تلمسها، فلو عملت Loop على الطلبات ولمست `order.Customer` ، هيحصل N+1 من غير ما تاخد بالك. وكمان محتاج Package اسمه Proxies و `virtual` على كل الـ Navigation Properties. فضّلت إني أطلب البيانات صراحة.

س14: ليه `Restrict` بين العميل والطلبات و `Cascade` بين الطلب والعناصر؟

ج: الطلبات سجل مبيعات مهم، فممنوع يضيع بسبب حذف عميل. قاعدة البيانات بترفض الحذف، والكود بيتأكد قبلها ويعرض رسالة واضحة. أما العنصر فملوش معنى من غير طلبه، فلما الطلب يتمسح، عناصره بتتمسح معاه.

س15: `SaveChangesAsync` بيعمل إيه بالظبط؟

ج: بيبص على كل الكائنات المتتبَّعة اللي اتغيرت (Added أو Modified أو Deleted)، ويحوّلها لـ `INSERT` و `UPDATE` و `DELETE`، وينفذهم كلهم جوه Transaction واحدة. بعد الـ `INSERT`، بيحط الـ `Id` الجديد في الكائن. وفي التعديل بيعمل `UPDATE` للأعمدة اللي اتغيرت بس. وفي زرار Generate، `RemoveRange` بيعلّم كل الصفوف إنها هتتمسح، و `SaveChangesAsync` بيبعت أوامر `DELETE`.

س16: الفرق بين `IQueryable` و `IEnumerable` ؟

ج: `IQueryable` بيبني استعلام لسه ما اتنفذش، وكل `Where` و `OrderBy` و `Skip` بيتحول لـ SQL ويتنفذ في قاعدة البيانات. والتنفيذ بيحصل عند `ToListAsync` أو `FirstOrDefaultAsync`. أما `IEnumerable` بعد ما البيانات توصل، فأي فلترة بتحصل في الذاكرة. عشان كده الـ Paging عندي متعمل قبل `ToListAsync`، فقاعدة البيانات بترجع 20 صف بس.

س17: ليه `decimal(18,2)` للسعر؟

ج: `decimal` دقيق في الأرقام العشرية، بعكس `double` اللي ممكن يدّي أخطاء تقريب صغيرة، وده مش مقبول في الفلوس. و `(18,2)` يعني 18 رقم منهم 2 بعد العلامة العشرية. حددتها بـ `HasPrecision(18, 2)`، لأن من غيرها EF بيطلّع تحذير ويستخدم إعداد افتراضي.

س18: إيه الـ Indexes اللي عملتها وليه؟

ج: Unique Index على `Email` عشان يمنع التكرار حتى لو طلبين وصلوا مع بعض. و Index على `Customers.Name` عشان ترتيب قائمة العملاء. و Index على `Orders.OrderDate` عشان "الأحدث الأول" مع الـ Paging. و EF عمل Index على كل Foreign Key (`CustomerId` و `OrderId`) عشان الـ JOINs.

أسئلة ASP.NET Core MVC

س19: يعني إيه PRG، وليه استخدمته؟

ج: Post/Redirect/Get. بعد POST ناجح بعمل `RedirectToAction` بدل ما أعرض View. كده لو المستخدم عمل Refresh، المتصفح بيعيد الـ GET مش الـ POST، فمفيش إضافة مكررة. والرسالة بتعدّي للصفحة الجاية في `TempData`.

س20: الفرق بين `TempData` و `ViewBag` و `ViewData` ؟

ج: `ViewData` قاموس بمفاتيح نصية، و `ViewBag` نفس الـ ViewData بس بشكل Dynamic (`ViewBag.Page`). الاتنين بيعيشوا لحد ما الـ View تترسم في نفس الـ Request. `TempData` بيعيش لحد الـ Request الجاي (متخزن في Cookie)، فينفع بعد الـ Redirect. عندي: `ViewBag` للـ Pager والـ Dropdown وعدد الطلبات، و `TempData` لرسايل النجاح والخطأ. وكمان `ViewData["Title"]` لعنوان الصفحة.

س21: Model Binding بيشتغل إزاي؟

ج: ASP.NET Core بياخد القيم من الفورم والرابط والـ Query String، ويحطها في الـ Parameters الـ Action بالاسم. مثلاً `name="Email"` يروح لـ `customer.Email`، و `/Customers/Details/5` يروح لـ `id`، و `?page=2` يروح لـ `page`. ولو القيمة مش متوافقة مع النوع (كلام في خانة رقم)، بيسجّل خطأ في `ModelState`.

س22: الـ Validation عندك فين؟

ج: 4 طبقات. Data Annotations في الـ Models بتتحقق في المتصفح (بـ jQuery Validation) قبل ما الفورم يتبعت. ونفس الـ Annotations بتتحقق على السيرفر تلقائي قبل الـ Action. وفحوصات زيادة في الـ Controllers للحاجات اللي محتاجة قاعدة البيانات أو تاريخ النهارده: إيميل متكرر، عميل مش موجود، تاريخ في المستقبل. وقيود قاعدة البيانات كآخر خط دفاع. ولو

`ModelState.IsValid` بـ false، برجّع نفس الفورم، والرسايل بتظهر بـ `asp-validation-for` و `asp-validation-summary`.

س23: إيه هو الـ Anti-forgery Token؟

ج: حماية من CSRF، يعني موقع تاني يخلّي متصفحك يبعت فورم لموقعي من غير ما تعرف. الـ Form Tag Helper بيحط Token مخفي في كل فورم POST، و `[ValidateAntiForgeryToken]` بيرفض أي POST من غيره.

س24: يعني إيه Overposting، وحميت منه إزاي؟

ج: إن حد يبعت حقول زيادة في الفورم، زي `Id` أو `OrderId`، عشان يغيّر حاجة مش مسموح بيها. حميت منه بـ `[Bind]` اللي بياخد الحقول المسموحة بس، وبطريقة Load-Copy-Save: بجيب السجل من قاعدة البيانات وبنسخ عليه الحقول المسموحة بس. وفي تعديل العنصر، `OrderId` بيتاخد من قاعدة البيانات مش من الفورم.

س25: لو المستخدم طلب سجل مش موجود؟

ج: الـ Action بترجع `NotFound()`، و `UseStatusCodePagesWithReExecute("/Home/HttpError", "?code={0}")` بيعرض صفحة "Not found" بنفس تصميم الموقع، والكود بيفضل 404. نفس الكلام لو الـ `id` مش رقم أو مش موجود في الرابط.

س26: إزاي بتتعامل مع الأخطاء غير المتوقعة؟

ج: في ASP.NET Core، Development بيعرض Developer Exception Page بالتفاصيل عشان المطوّر. في Production، `UseExceptionHandler("/Home/Error")` بيعرض صفحة عامة فيها `RequestId` بس. وأخطاء الحفظ بمسكها بـ `catch (DbUpdateException)` وبسجّلها بـ `ILogger`، وبعرض رسالة بسيطة للمستخدم.

س27: ليه `async/await` ؟

ج: عشان الـ Thread ما يفضلش واقف مستني قاعدة البيانات. وقت الانتظار بيخدم Requests تانية، فالسيرفر بيستحمل مستخدمين أكتر بنفس الموارد. كل methods قاعدة البيانات عندي Async.

س28: الـ Paging عملته إزاي؟

ج: `CountAsync` عشان أحسب عدد الصفحات، و `Math.Clamp` عشان رقم الصفحة يفضل في الحدود، وبعدين `Skip((page - 1) * 20).Take(20)` قبل `ToListAsync`، فبيتحولوا لـ `OFFSET/FETCH` في SQL. وزرايز Previous و Next في Partial View اسمها `_Pager` مستخدمة في الصفحتين.

س29: يعني إيه Layout و Partial View و Tag Helper؟

ج: الـ Layout (`_Layout.cshtml`) هو الإطار المشترك (الـ Sidebar والرسايل وملفات CSS)، وكل صفحة بتتحط مكان `@RenderBody()`. الـ Partial View جزء بيتعاد زي `_Pager`. والـ Tag Helpers زي `asp-action` و `asp-for` و `asp-validation-for` بيولّدوا HTML صح على السيرفر: روابط، وأسماء حقول، ورسايل خطأ.

أسئلة الأداء

س30: صفحة الأداء بتقيس إزاي؟ وإيه اللي بيخلي القياس عادل؟

ج: الطرق التلاتة بتبني نفس النتيجة بالظبط (`List<OrderListItem>`)، والصفحة بتتأكد إن البيانات متطابقة. كل طريقة ليها تشغيلة تسخين مش بتتقاس، وبعدها عدد التشغيلات المطلوب. قبل كل تشغيلة بعمل `ChangeTracker.Clear()` عشان ما تستفيدش من بيانات اللي قبلها، والوقت بيتقاس بـ `Stopwatch`. البيانات التجريبية ثابتة بـ `Random(12345)`، وكل نتيجة بتتسجل في CSV ومعاها حجم البيانات وعدد التشغيلات، عشان أقارن نتايج تحت نفس الظروف.

س31: عرفت عدد الاستعلامات إزاي؟ وعرضت الـ SQL إزاي؟

ج: في N+1، الكود بيعدّ بنفسه: 1 للاستعلام الأول، وبيزوّد 1 مع كل `LoadAsync` للعناصر، ومع كل عميل لسه ما اتحمّلش (بسأل `Entry(order).Reference(o => o.Customer).IsLoaded`). و `Include` و `Select` بيعملوا استعلام واحد بس. والـ SQL بجيبه بـ `ToQueryString()` ، اللي بترجّع نص الـ SQL من غير ما تنفذه، وبعرضه في الصفحة.

س32: إيه النتايج، وإيه القرار اللي خدته؟

ج: على 200 طلب: N+1 حوالي 431 ms و 281 استعلام، و Include (مع `AsNoTracking`) حوالي 28 ms باستعلام واحد، و Select حوالي 6 ms باستعلام واحد. القرار: Select لقائمة الطلبات، و Include + AsNoTracking للتفاصيل، ومفيش N+1 في أي قائمة.

أسئلة التصميم والتحسين

س33: ليه مفيش Repository أو Service Layer؟

ج: لأن المشروع صغير، و `DbContext` أصلاً بيعمل شغل الـ Repository (الـ `DbSet`) والـ Unit of Work (`SaveChanges`). ولو المشروع كبر أو احتجت Unit Tests للـ Logic، هعمل Services (مثلاً `OrderService`) وأخلّي الـ Controllers تكلمها بدل الـ DbContext.

س34: لو هتضيف Login؟

ج: ASP.NET Core Identity لتسجيل الدخول وجداول المستخدمين، و `[Authorize]` على الـ Controllers، و Roles، مثلاً Admin بس يقدر يمسح أو يدخل صفحة الأداء. وكمان زرار Generate لازم يتشال أو يتقفل في Production.

س35: لو عندك مليون طلب، هتحسّن إيه؟

ج: الـ Paging والـ Projection والـ Index على `OrderDate` شغالين كويس فعلاً. بس `OFFSET` بيبطأ في الصفحات البعيدة جداً، فممكن أستخدم Keyset Pagination (أجيب الصفحة بعد آخر تاريخ ورقم شفتهم). والـ Dropdown بتاع العملاء محتاج بحث (Autocomplete) بدل تحميل الكل. والـ `CountAsync` ممكن يتعمله Cache.

س36: إيه أصعب مشكلة قابلتك؟

ج: (اختار واحدة) مثلاً صفحة الطلبات طلّعت "The view 'Index' was not found"، والسبب إن فولدر `Views/Orders` اتنقل بالغلط جوه `Views/OrderItems` . فهمت من رسالة الخطأ الأماكن اللي MVC بيدور فيها على الـ View، ورجّعت الفولدر. أو مشكلة N+1: الأرقام في صفحة الأداء أثبتت الفرق بوضوح، وغيّرت بسببها طريقة تحميل القائمة.

س37: لو هتبدأ المشروع من الأول، هتعمل إيه مختلف؟

ج: كنت هعمل View Models منفصلة للفورمز بدل ما الفورم يستخدم الـ Entity على طول، وأضيف Unit Tests، و Concurrency Token، و Authentication من الأول.

س38: لو اتنين عدّلوا نفس السجل في نفس الوقت؟

ج: حالياً آخر واحد يحفظ هو اللي بيكسب، لأن مفيش Concurrency Token. الحل إني أضيف عمود `RowVersion` (`[Timestamp]` أو `IsRowVersion()` في Fluent API). ساعتها EF يرمي `DbUpdateConcurrencyException` لو السجل اتغير بعد ما المستخدم فتحه، وأعرض له رسالة.

س39: إزاي المشروع محمي من SQL Injection و XSS؟

ج: من SQL Injection: EF Core بيحط القيم كـ Parameters في الاستعلامات، ومفيش SQL مكتوب بإيد في المشروع. ومن XSS: Razor بيعمل HTML Encoding تلقائي لأي قيمة بتتعرض بـ `@` .

س40: إيه فايدة `[Display]` و `[DataType]` ؟

ج: `[Display(Name = "Unit price")]` بيغيّر النص اللي بيظهر في الـ Label وفي رسايل MVC الجاهزة، بدل اسم الـ Property `UnitPrice` . و `[DataType(DataType.Date)]` بيخلّي الـ Tag Helper يعمل `<input type="date">` ، فالمتصفح بيعرض Date Picker من غير وقت. الاتنين ملهمش أي تأثير على قاعدة البيانات.

16. ملحق: قاموس المصطلحات وأوامر مهمة

16.1 قاموس المصطلحات

المصطلح	المعنى باختصار
Action	method عامة في Controller بترد على رابط
Anti-forgery Token	Token مخفي في الفورم بيحمي من CSRF
`AsNoTracking`	قراءة من غير ما EF يتابع الكائنات (أسرع)
Async / await	انتظار من غير ما الـ Thread يقف
Cascade Delete	حذف الأب بيحذف الأبناء تلقائي
CDN	سيرفرات عامة بتقدّم ملفات المكتبات
Change Tracker	الجزء في DbContext اللي بيعرف إيه اللي اتغير
`ChangeTracker.Clear()`	بتخلّي الـ DbContext ينسى كل الكائنات اللي حمّلها
Connection String	عنوان قاعدة البيانات وطريقة الدخول ليها
Controller	كلاس بيستقبل الطلبات وينسّق بين البيانات والعرض
Conventional Routing	Routing بشكل رابط ثابت للمشروع كله
CRUD	Create و Read و Update و Delete
CSRF	هجوم بيخلّي متصفحك يبعت طلب من غير ما تعرف
Data Annotations	Attributes فوق الـ Properties للتحقق، زي `[Required]` و `[Range]`
`DbContext`	جلسة الشغل مع قاعدة البيانات في EF
`DbSet<T>`	بيمثل جدول
Deferred Execution	الاستعلام بيتبني ومش بيتنفذ إلا عند `ToList` وأمثالها
Dependency Injection	الكلاس بياخد اللي محتاجه من برة بدل ما يعمله
Eager Loading	تحميل البيانات المرتبطة مع الأصل (`Include`)
Entity	كلاس بيمثل جدول
Explicit Loading	تحميل البيانات المرتبطة بأمر صريح بعدين
Fluent API	تعريف قواعد الجداول بالكود في `OnModelCreating`
Foreign Key	عمود بيشاور على Primary Key في جدول تاني
Identity	رقم بيتولّد تلقائي في SQL Server
Index	فهرس بيسرّع البحث والترتيب
Layout	الإطار المشترك لكل الصفحات
Lazy Loading	تحميل تلقائي وقت ما تلمس الـ Property (مش مستخدم)

المصطلح	المعنى باختصار
LINQ	استعلامات مكتوبة بـ C#
Middleware	خطوة في سلسلة معالجة الطلب
Migration	ملف بيسجّل تغيير في شكل قاعدة البيانات
Model Binding	تحويل بيانات الطلب لـ Parameters و Objects
`ModelState`	نتيجة التحقق ورسايل الأخطاء
N+1	استعلام + استعلام لكل سجل (بطيء)
Navigation Property	Property بتشاور على Entity مرتبطة
NuGet	مخزن مكتبات .NET.
Nullable Reference Types	ميزة في C# بتحذّرك من الـ `null`
ORM	مكتبة بتربط الكلاسات بالجداول
Overposting	إرسال حقول زيادة في الفورم
Paging	تقسيم القائمة لصفحات
Partial View	جزء View بيتعاد استخدامه
Pipeline	سلسلة الـ Middleware بالترتيب
PRG	Post/Redirect/Get
Primary Key	العمود اللي بيميّز كل صف
Projection	اختيار أعمدة معينة بـ `Select`
Razor	لغة بتخلط HTML و C#
Restrict	منع حذف الأب لو ليه أبناء
Routing	ربط الرابط بالـ Controller والـ Action
Scoped	نسخة لكل Request
Seeding	إضافة بيانات أولية
Split Query	تقسيم استعلام Include لأكتر من استعلام
Tag Helper	Attributes بتبدأ بـ `asp-` وبتولّد HTML
`ToQueryString()`	بترجّع نص الـ SQL اللي EF هيبعته، من غير ما تنفذه
`TempData`	بيانات بتعيش لطلب واحد بعد الـ Redirect
Transaction	مجموعة عمليات يا تنجح كلها يا تتلغي كلها
Unique Index	Index بيمنع تكرار القيمة
Validation	التحقق من صحة البيانات
View	صفحة `.cshtml` بتعرض البيانات

المصطلح	المعنى باختصار
View Model	كلاس مصمم لصفحة معينة، مش جدول
`ViewBag`	بيانات زيادة من الـ Controller للـ View
XSS	حقن سكريبت في الصفحة

16.2 أوامر مهمة

تشغيل المشروع:

```
dotnet run --project ExamRFID/ExamRFID
```

(أو من Visual Studio: F5. البروفايل `https` على `https://localhost:7294`.)

الـ Migrations من Package Manager Console (اكتب اسم التعديل بتاعك مكان `AddCustomerAddress`):

```
Add-Migration AddCustomerAddress
Update-Database
Remove-Migration
```

نفس الأوامر من الـ Terminal (جوه فولدر المشروع `ExamRFID/ExamRFID`):

```
dotnet ef migrations add AddCustomerAddress
dotnet ef database update
dotnet ef migrations remove
dotnet ef migrations has-pending-model-changes
```

البيانات اللي بتتعمل أول مرة: 100 عميل. ولو عايز بيانات أكتر للاختبار، استخدم زرار **Delete all and generate** في صفحة Performance (من 10 لـ 5000 عميل).

16.3 إزاي تعمل ملف PDF تاني بعد ما تعدّل التوثيق

ملف `ProjectDocumentation.md` هو الأصل اللي تعدّل فيه. بعد التعديل، تقدر تعمل الـ PDF تاني بالسكريبت `build_pdf.py` اللي في نفس الفولدر:

```
pip install markdown pygments
python Documentation/build_pdf.py
```

السكريبت بيحوّل الـ Markdown لـ HTML باتجاه من اليمين للشمال، وبعدين Microsoft Edge بيطبعه PDF. محتاج Microsoft Edge متثبت (موجود في Windows)، ونت عشان خط IBM Plex Sans Arabic بيتحمّل من Google Fonts.